



Notes

Feb 27, 2026

Causal Inference Workshop Auto1

Day1

Invited [Martin Bierey](#) [Ling-Chia Chen](#) [Philipp Bach](#) [Chieri Naito](#) [Leon Ostermann](#)
[tsz.lam@auto1.com](#) [Khalil Mlayhi](#) [Silvia Messuti](#) [Ani Pole](#) [Javier Otero Marquez](#)
[Alena Nazarava](#) [Muhammad Rizvi](#) [Daniel Jacob](#) [Nicolas Pardo](#) [João Brazuna](#)
[Yana Savva](#) [zahra.mokhtari@auto1.com](#) [Bojana Mirkovic](#)
[derya.durmush@auto1.com](#) [Dr. Sven Klaassen](#) [Jan Rabenseifner](#)
[Prof. Dr. Martin Spindler](#) [Ruizhe Chao](#) ~~[Haneer Rai](#)~~

Attachments [📎 Causal Inference Workshop Auto1 Day1](#)

Meeting records [📄 Transcript](#)

Summary

The economic AI team, including Sven Klaassen, Jan Teichert-Kluge, Jan Rabenseifner, Martin, and Philip Bach, outlined the meeting schedule and the core goal of merging machine learning and causal inference, specifically focusing on the Double Machine Learning (DML) framework for valid causal effect estimation. Key DML components—Neyman orthogonality, high-quality machine learning estimation, and cross-fitting—were discussed to address the regularization bias introduced by using machine learning models for nuisance functions, which Tsz Yan Lam, Martin Bierey, and Daniel Jacob inquired about in relation to model structure, binary outcomes, and hyperparameter tuning. The participants were encouraged to download the first notebook for the post-lunch lab session, which is designed to provide hands-on experience with the DML workflow, including model fitting, evaluation, and the crucial role of hyperparameter tuning with Optuna.

Details

- **Meeting Kickoff and Introductions:** The meeting started slightly after the scheduled time, following a brief delay to await more colleagues. The session began with a short round of self-introductions from the economic AI team to set the stage for the meeting ([00:00:00](#)). Sven Klaassen, a software development expert and maintainer of the double ML package, introduced himself first, followed by Jan Teichert-Kluge and Jan Rabenseifner, who are both involved in research and applied aspects of causal inference at the chair and economic AI ([00:04:59](#)).
- **Team Roles and Backgrounds:** The economic AI team's expertise was outlined, including Martin, a co-founder of economic AI and a leader in causal inference. Philip Bach, a professor, manages the training program and led the development of the R package for double ML. Sven Klaassen was highlighted as the primary architect of the Python double ML package ([00:13:57](#)).
- **Overview of Today's Schedule:** The day's schedule includes an introduction to causal machine learning and double ML, followed by a break and a discussion on hyperparameter tuning. A lunch break will provide time to review the first notebook, which is on returns of investments. The afternoon will feature two separate notebook sessions and a discussion on treatment effects, concluding with a Q&A session ([00:07:43](#)).
- **Overview of Next Week's Schedule:** The second day, scheduled for the following week, will include a recap and use case session, a deep dive into the difference-in-differences approach, a second notebook and lab session, and a discussion on sensitivity analysis ([00:07:43](#)). All slides and notebooks are available for download via a shared link, which Sven placed in the chat ([00:09:07](#)).
- **Engagement and Preparation Expectations:** Participants were encouraged to use the lunch break to download and become familiar with the first notebook so that the following notebook session can be an interactive discussion ([00:09:07](#)). The presenters also emphasized that questions and comments should be shared interactively during the presentations, either by unmuting or using the chat, to promote a dynamic discussion ([00:10:30](#)). Due to the size of the attending team, individual introductions from all participants were deferred until the smaller lab sessions ([00:11:41](#)).

- Merging Machine Learning and Causal Inference:** The core goal is to merge the fields of machine learning and causal inference, known as causal machine learning. Machine learning is noted for its success in learning complex patterns and its forecasting power, typically based on correlations and associations ([00:15:13](#)). Causal inference focuses on learning causal relationships, moving beyond mere correlations, which is essential for "real intelligence" ([00:16:52](#)).
- Causality vs. Prediction and Applications:** Predictive modeling aims to build strong prediction rules for an outcome Y based on features X , while causal modeling focuses on determining the causal effect of a treatment D on an outcome Y ([00:21:08](#)). Causal machine learning is used in various applications, including resource allocation, A/B testing, uplift modeling, dynamic pricing, clinical trials, and product optimization ([00:18:20](#)).
- Fundamental Problem of Causal Inference:** Causal inference typically begins by defining the causal parameter of interest and stating necessary assumptions for identification and valid estimation ([00:25:28](#)). The fundamental problem is that individual causal effects cannot be generally identified because only one potential outcome (the factual) is observed, while the other (the counterfactual) is not ([00:32:46](#)).
- Identification vs. Estimation:** The process of causal inference moves from a hypothetical causal estimate to a statistical estimate, then to a modeling estimator, and finally to a statistical estimate that provides actionable insights ([00:26:55](#)). The core challenge, identification, is about ensuring that the assumptions are plausible so that observations in the data can be generalized to hypothetical quantities ([00:28:24](#)).
- Potential Outcomes and Average Treatment Effects (ATE):** Using uplift modeling (email coupons' effect on sales) as an example, the binary treatment D leads to two potential outcomes, $Y(1)$ and $Y(0)$, for a treated and untreated subscriber ([00:31:01](#)). While the individual causal effect $Y(1) - Y(0)$ is unidentifiable, the Average Treatment Effect (ATE) can often be identified, which is the difference in expected outcomes if everyone were treated versus if no one were treated ([00:32:46](#)).
- Randomized Control Trials (RCTs) and Observational Data:** In an ideal RCT, treatment assignment is randomized and independent of potential outcomes, allowing the ATE to be consistently estimated by comparing the observed outcomes of the treatment and control groups ([00:34:19](#)). However, when RCTs

are not possible due to constraints, observational data is used, necessitating the "selection on observables" (unconfoundedness) assumption to estimate causal effects ([00:37:28](#)).

- **Addressing Confounding in Observational Data:** With observational data, treatment assignment is not random, and confounding factors X influence both the treatment D and the outcome Y . The unconfoundedness assumption posits that if they condition on a rich set of observed covariates X , the treatment becomes "as good as random," allowing for an apples-to-apples comparison of individuals with similar characteristics ([00:39:00](#)) ([00:44:27](#)).
- **Initial Causal Estimation Methods:** Methods for causal estimation when controlling for confounders include regression adjustment, reweighting approaches using inverse propensity scores, and doubly robust methods that combine both ([00:44:27](#)). Simple linear regression can be used for causal estimation, but it requires strong assumptions like linearity and an additive model, in addition to correctly identifying the variables to condition on ([00:40:19](#)).
- **Need for Double Machine Learning (DML):** When dealing with high-dimensional problems, machine learning methods are needed because linear regression performs poorly, and regularization techniques like Lasso introduce bias into the treatment effect estimates ([00:43:08](#)) ([00:47:21](#)). DML is necessary to leverage the predictive power of machine learning algorithms while maintaining valid inference in the presence of regularization ([00:47:21](#)) ([00:49:47](#)).
- **Machine Learning for Causal Inference:** Machine learning methods are powerful for modeling the complex relationships between the outcome Y , the treatment D , and the covariates X , moving beyond the limitations of simple linear regression ([00:49:47](#)). There was a discussion regarding the importance of selecting the correct variables to control for, as improperly controlling for variables can downgrade performance ([00:51:06](#)).
- **Double Machine Learning Framework:** Double machine learning merges modern predictive algorithms (like Lasso, Random Forest, or boosting methods) with the robust inferential tools of econometrics and statistics ([00:52:31](#)). This framework aims to achieve high predictive power and valid inference, where the predictive power of the machine learning methods is what matters for the asymptotics of the point estimate ([00:56:27](#)).
- **Nuisance Functions and Causal Estimation Bias:** Nuisance functions, defined as outcome regression (conditional expectation of outcome given features) and

propensity score estimates (probability of treatment given covariates), are typically estimated using machine learning, but plugging these predictions directly into the treatment effect estimation introduces bias. Machine learning methods utilize regularization (e.g., penalization or early stopping) to optimize for prediction accuracy, which introduces regularization bias into the nuisance estimates, consequently propagating to the causal estimate, rendering the naive approach ineffective ([00:58:44](#)). Small errors in nuisance estimates can lead to substantial bias in the causal estimate, which invalidates standard inference methods such as confidence intervals and p-values ([01:00:08](#)).

- **Neyman Orthogonality for Bias Reduction:** A framework is needed to immunize the causal estimate against regularization bias, and the first key ingredient of Double Machine Learning (DML) is Neyman orthogonality, which constructs score functions such that errors in nuisance functions only have second-order effects on the causal estimates ([01:00:08](#)). The geometric intuition is that the estimation error in the nuisance direction (horizontal axis) does not change the position of the causal parameter (vertical axis) because they are set up to be orthogonal to each other. The double machine learning package automatically uses orthogonal scores, but they must be derived when extending the package with new models ([01:02:48](#)).
- **High-Quality Machine Learning Estimation:** The second ingredient for DML is the requirement for high-quality machine learning estimation, as orthogonality only reduces bias to some degree, meaning nuisance errors cannot be arbitrarily large. The product of machine learning errors from the different nuisance components (outcome regression and propensity score estimation) must still shrink fast enough. Practical guidance suggests using flexible learners appropriate for the data and performing proper hyperparameter tuning for nuisance functions, as the impact of tuning is substantial ([01:04:34](#)).
- **Sample Splitting and Cross-fitting:** The third DML ingredient is sample splitting, which prevents overfitting done during training from contaminating the causal estimates by ensuring that overfitting errors are not correlated with estimation errors. The implementation solution is cross-fitting, where data is split into K folds (e.g., five), and for each fold, machine learning methods are trained on the other folds to make held-out predictions ([01:07:38](#)). The benefits of cross-fitting are that predictions are always out-of-sample, and all data is used for both training and estimation, which is more efficient than a simple train-test split ([01:09:05](#)).

- Main Result and Implementation Overview:** When combining the key ingredients—Neyman orthogonality, high-quality machine learning estimation, and sample splitting—the DML estimator is consistent, and the sampling error is approximately normally distributed, which enables valid inference and the construction of standard errors and confidence intervals ([01:10:19](#)). The package implementation is object-oriented, exploiting a common structure around score functions, which facilitates flexibility in combining data and models ([01:11:54](#)). The package is built to easily use state-of-the-art machine learning predictions and tuning methods for nuisance components, utilizing a similar API to scikit-learn, and the default setting for sample splitting is five-fold cross-fitting ([01:13:09](#)).
- Implemented Model Types:** The DML package has a constantly growing list of implemented models, including the general class of partially linear models, nonparametric interactive regression models, potential outcome models, instrumental variables, and recently added difference in differences, sample selection, and regression discontinuity ([01:14:25](#)) ([01:24:59](#)). The models support various causal estimates, such as average treatment effect, conditional average treatment effect (CATEs), and group average treatment effects ([01:15:50](#)). Martin Bierer asked if group average treatment effects require predefined groups, and Sven Klaassen clarified that most implementations rely on predefined groups, although some methods may attempt to find groups using tree-based approaches ([01:23:29](#)).
- Treatment and Outcome Restrictions:** Tsz Yan Lam inquired about restrictions on the outcome type (continuous or binary) for the implemented models. Sven Klaassen noted that most models are applicable to either binary or continuous outcomes, but all settings assume a single outcome, not a multivariate one ([01:18:27](#)). For the partially linear model with a binary outcome, a logistic version is implemented, as an additive probability model may not be appropriate, suggesting that nearly all models should accommodate continuous or binary outcomes ([01:19:33](#)).
- Handling Binary Outcome Residuals:** Tsz Yan Lam asked how residuals are defined in the case of a binary outcome (0 or 1), given that standard residual-on-residual regression relies on a continuous target ([01:19:33](#)). Sven Klaassen explained that the residual-on-residual intuition comes from the orthogonality assumption in the partially linear regression model, which relies on an additive structure ([01:20:44](#)). For binary outcomes, the logistic version or a

completely interactive model is often more appropriate, and in those cases, the method changes to a plug-in score function rather than residual-on-residual regression, maintaining the orthogonality property ([01:22:13](#)).

- **Typical DML Workflow Steps:** The typical DML workflow starts with problem formulation, including drawing a dependence graph (DAG) to visualize available and unobservable features, which helps in interpreting and getting a feeling for the problem ([01:40:43](#)). Next, users define data objects by specifying the outcome (Y), treatment (D), and conditioning variables (X) ([01:43:36](#)). The third step is model selection, where users choose an appropriate DML model (e.g., partially linear, interactive regression) based on the desired estimand and data structure ([01:45:03](#)).
- **Defining Machine Learning Estimators:** The fourth step in the DML workflow is defining the machine learning methods (learners) to be used for the nuisance functions, which must follow the scikit-learn API style with fit and predict methods ([01:45:03](#)). Users must choose one model for the outcome regression (G) and one model for the treatment regression/classification (M), where the choice depends on the outcome and treatment types ([01:47:39](#)). Daniel Jacob asked about consistency between the outcome and treatment regressors and Sven Klaassen clarified that in their implementation, one feature vector (X) is typically used for both regressions ([01:50:20](#)).
- **Hyperparameter Tuning and Prediction Accuracy:** The final steps in the workflow involve using the `tune_models` method to perform hyperparameter tuning to achieve the best cross-fitted predictive performance for both the treatment and outcome predictions ([01:55:37](#)). Martin Bierey asked about accuracy goals for both models, and Sven Klaassen confirmed that the aim for both models is high predictive accuracy, noting that for the outcome model, the treatment variable should often be included in the regression ([01:54:31](#)). Tsz Yan Lam confirmed that models without a scikit-learn interface could be wrapped around with the required fit and predict functions, or external predictions could be used in complex cases ([01:49:13](#)).
- **API Integration and Model Fitting:** The discussion focused on running models with an API design that is close to the scikit-learn (sklearn) style, where the process involves setting up the learner and problem, and then using a `fit` method. The `fit` method internally handles cross-fitting and runs machine learning models after hyperparameter setting, resulting in a final point estimate and standard errors. The point estimate for a random forest example was 8.135,

noting that standard errors in these problems are typically large due to measurement noise ([01:56:56](#)).

- **Predictive Performance and Model Summaries:** The main idea is that the predictive performance of the models should matter in the background to estimate conditional expectations or probabilities that solve the underlying problems. Models that perform well yield predictions that are close to each other, resulting in very close final point estimates. The final object includes a summary detailing the number of observations used, the type of machine learning algorithm, and the corresponding out-of-sample performance ([01:58:27](#)).
- **Confidence Intervals and Evaluation Options:** The object summary provides standard statistical outputs, including standard errors, t-values, p-values, and confidence intervals (CIs). Users can create CIs at different levels, which are calculated via a normal approximation within the framework ([01:59:55](#)). It is also possible to include jointly valid CIs using a bootstrapping procedure for scenarios with multiple treatment effect estimates ([02:01:10](#)).
- **Hyperparameter Tuning with Optuna:** A recent addition to the package is the integration of the Optuna tuning framework, which is flexible and customizable ([02:01:10](#)). The user's main task is to achieve good predictive performance, as the cross-fitting and score orthogonality are already implemented in the model. Tuning is crucial because default settings, such as those sometimes found in sklearn's random forest, can lead to overfitting, making hyperparameter selection relevant to minimize nuisance errors and improve generalization ([02:02:29](#)).
- **Defining Hyperparameter Search Space:** To perform hyperparameter tuning with Optuna, users must define ``trial`` objects, which specify the ranges for hyperparameters to be tested. This involves defining a function that returns a dictionary matching the desired hyperparameters ([02:04:09](#)). The search spaces, which can include floating, integer, or categorical values, must be defined separately for treatment and outcome regression, though they can be reused if identical ([02:05:42](#)).
- **Optimization Process and Model Comparison:** The ``tune_models`` method is run with defined hyperparameter spaces and settings, such as the number of trials. Optuna employs an optimization process that samples hyperparameter configurations, balancing exploration with sampling near good combinations, which is more optimized than random grid search ([02:07:59](#)). A comparison between an untuned and tuned LightGBM model showed the estimate changing

drastically (from 0.06 to 0.5), with the tuned model yielding a much smaller standard error, demonstrating the impact of tuning ([02:10:36](#)) ([02:19:16](#)).

- **S-Learner vs. T-Learner Structures:** The framework uses a T-learner structure for interactive regression, which fits two separate outcome regressions: one for the treatment group and one for the control group ([02:10:36](#)). A T-learner is considered more flexible than an S-learner (single regression including treatment as a feature), as it is equivalent to including all interaction terms between treatment and features ([02:11:55](#)) ([02:15:26](#)). The framework fits these two models separately by cloning the outcome regression model, which allows for potentially different hyperparameter combinations to address issues like unbalanced treatment assignment ([02:13:03](#)).
- **Derivation of Standard Errors:** Standard errors are derived using a Wald-type approach based on the score function and an asymptotic representation of the estimator ([02:20:41](#)). Specifically, the computation involves calculating the second moment of the scores empirically along with a necessary scaling factor ([02:21:51](#)). Testing has shown that the coverage for these standard errors performs well ([02:23:12](#)).
- **Evaluating Learner Performance:** The `evaluate_learners` method allows users to assess the cross-fitted performance of the learner using metrics like root mean square error (RMSE) for continuous variables or log loss for binary variables ([02:23:12](#)). Out-of-sample performance metrics improved significantly after tuning, explaining why the causal effect estimation was better ([02:24:27](#)).
- **Using Scikit-learn Pipelines:** The package supports the use of scikit-learn pipelines, allowing users to combine multiple regressors (e.g., linear regression and boosting) and incorporate steps like robust scaling before model fitting ([02:24:27](#)). This pipeline structure works within the tuning framework, enabling the separate tuning of components, which is helpful for creating robust estimations by combining different types of models ([02:27:07](#)).
- **Accessing Detailed Tuning Results:** Users must set a flag to return detailed tuning results, which go beyond modifying the object with optimal hyperparameters. The results object contains specific tuning results for each regression (e.g., outcome control, outcome treated, and propensity score), including the best score achieved, chosen parameters, and individual trial data ([02:28:30](#)).

- **Feature Transformation and External Integration:** The discussion touched upon complex feature transformations, such as the target encoding used by CatBoost, and their integration into the package ([02:31:35](#)). While it may not be straightforward if the transformation does not fit the standard pipeline structure, users can generate predictions externally and supply them to the model, treating the external estimate as if it were computed internally ([02:34:47](#)).
- **Next Steps and Session Preparation:** The plan is to transition to the lunch break, and participants are encouraged to download the provided notebook and review its contents ([02:38:05](#)). After the break, the sessions will involve working through the case study in groups, with the expectation that participants will run the notebook themselves to better understand the steps and potential effects of changes ([02:39:04](#)).

Suggested next steps

- ☐ Sven Klaassen will add an allowed outcome column to the table listing the current models implemented in the package.
- ☐ Tsz Yan Lam will create breakout rooms to assign people relatively randomly into different groups after the lunch break.

You should review Gemini's notes to make sure they're accurate. [Get tips and learn how Gemini takes notes](#)

Please provide feedback about using Gemini to take notes in a [short survey](#).



Transcript

Feb 27, 2026

Causal Inference Workshop Auto1

Day1 - Transcript

00:00:00

Alena Nazarava: Hello.

Tsz Yan Lam: I think we are still waiting for a number of our colleagues joining

Sven Klaassen: So, so for Suso, so um he he will also just join I think now for the um introduction part. Okay.

Jan Rabenseifner: Morning.

Sven Klaassen: Good

Daniel Jacob: Morning.

Tsz Yan Lam: Morning.

Sven Klaassen: morning.

Jan Teichert-Kluge: Good

Ani Pole: Morning.

Tsz Yan Lam: Okay, I think we are 1 minute away from the original time. Let me ping the team and if they are still not joining, I think we should just start.

Jan Rabenseifner: Sounds

Sven Klaassen: Okay.

Jan Rabenseifner: good.

Tsz Yan Lam: All right, I think we should just start. Um, so thank you very much for everyone joining here. I think I have met Yan, uh, but not everyone from your team. Uh, so how should we do it?

Sven Klaassen: Um yeah. Hi. Um so so I I think may maybe um a very very short introduction round um like and and then um then I can say something about the short of about the schedule and then we can get right started.

00:04:59

Sven Klaassen: I think would that be I think a good idea, right? this um okay may then let's just let me start um so so I'm Sven um I will be doing large part of the um the the

sessions on the present the ideas and the

Jan Teichert-Kluge: Okay.

Sven Klaassen: the main um problems and I'm currently so I'm working at economic AI specifically as in in the part of software development sort of and um the maintainer of the double ML package actually. So I'm currently like like what what is developed and have sort of the idea what what do we want to include and I've I've worked previously at Martin's chair so so Martin Schwintler for for this um to to actually did some research in this specifically in the field of um causal causal inference at this part um okay maybe I can then so so just shortly pass uh to Yan um Tika Okay.

Jan Rabenseifner: Yeah.

Jan Teichert-Kluge: Yes, thank you.

Sven Klaassen: So,

Jan Teichert-Kluge: Um, yeah, nice to see you. Uh, of course, um, my name is Jan. Um, I'm also based in Hamburg.

00:06:16

Jan Teichert-Kluge: Um, and I work for the chair, Professor, Professor Schlintler. Um, but also for economic AI. Um, so it's really nice to have the research side at the university and some more applied uh, insights from economic AI. So I'm really happy to be here and um yeah I will do uh one of the notebook sessions later.

Jan Rabenseifner: Yes, then I take over. Um, so um yeah, I'm also young and uh I'm uh working for economic AI now for three years. worked as a senior data scientist and um recently um also took over the organization of our consultancy projects and um yeah I also work in the do some research in the field of causal inference causal machine learning uh for some years now and um also just handed in my dissertation so I'm in a good mood right now and yes um and I will also do the first um session and also um do one of the to lab sessions later in the afternoon. Then um maybe I start sharing my screen. So um I hope in a second you can all see.

00:07:43

Jan Rabenseifner: So um yes regarding um the schedule of the day um so we start with a general introduction to cause and machine learning and WML h then we have a um break and afterwards we'll discuss um some hyperparameter tuning and um then during

the lunch break you will have some additional time to take a first look at the notebook one the notebooks are also included on this site. If you just scroll downwards a bit. Um so the first one is on returns of um investments. Yeah. And um we also have a um then the notebook session afterwards where we discuss discuss it together. So two separate sessions uh one by Yan and one by myself. And uh then after a break we'll discuss uh treatment effect here quite popular field I would say and uh we finish by some uh Q&A and also closing that's the plan for today and um then on the second day um that will be in one week uh we have a a short uh recap and use case session uh then we uh discuss the very famous difference and differences approach uh probably a lot of you are already familiar with that and we will dive deeply into that topic.

00:09:07

Jan Rabenseifner: H then we have a a second notebook and um our second lab session and then we uh finish uh with sensitivity analysis and again our Q&A session and closing. H this website here is also available. Um I think Sven will put it in the in the chat so you can all have a look and download for example the the notebooks um so you can prepare them later and all slides that we discuss they're also included here. Yeah just shared it. Thanks. All right. Any additional points that I missed? Sven you're unmute yourself.

Sven Klaassen: Um yeah, ju just as a very short comment. So so the lunch break is essentially of course it's it's a bit longer, right? So, so it would be really nice like for for you to to have like and um already specifically download the notebook and open it and have a look on it to to see whether you can execute or run of the some of these parts parts already make yourself a bit familiar because like um on the the notebook session it should be more or less like a discussion right uh where we try to go through it together and maybe then you have already some comments specifically What of course we we have something to tell right some something to show on this but then you have some

00:10:30

Sven Klaassen: comments to make it a bit more interactively right in in this specific part and if you want to um you download it just um just right click on the notebook and save link as um then you you it will automatically download the whole Jupyter notebook for

for for for both of these sessions. So, so if there are some problems just of course just tell us and during the the the the general sessions and the presentations I the the idea that is that it should be somewhat interactive right. So, so, so if you have um some questions and you or open points or you want to share some points from your use case and things like this um just try just unmute yourself, ask us um if it gets too much of course we we might delay some parts in later sessions essentially but but um I I think it's it's always better to have a discussion on the current topic um and to have it a bit more interactively than just like like listening to to what we are telling. So, so if so, so so so then that would be great.

00:11:41

Sven Klaassen: I I I'm also like like there will also be always one person who's looking at the chat. So, so you can also try to to ask things in the chat and we we will try to answer there. But this sometimes also takes more time and not everybody sees it. So yeah, just just as a general overview.

Jan Rabenseifner: Yes.

Sven Klaassen: Okay.

Jan Rabenseifner: Great.

Tsz Yan Lam: All right. Do you expect us to introduce ourselves or we should just go along?

Jan Rabenseifner: Thanks.

Sven Klaassen: I

Jan Rabenseifner: I think you're such a large team,

Sven Klaassen: mean,

Jan Rabenseifner: but um I don't know how you normally do it. Um uh I would say um the in the lab session we have maybe a bit more time and we have two separate groups and then maybe let's do it in the smaller group. I think that's fine by every everyone. All

Sven Klaassen: I I it like depends a bit on on for for yourself if you all I think you all know each other right?

00:12:31

Jan Rabenseifner: right.

Sven Klaassen: So so that that would be a lot large part there. So but you we can also

say like you just say just very short sentence on on something and then we make a short round. So so we we at least know everybody. Yeah. I don't know. Yeah. Okay. Or we make it in lab session. I think it's easier.

Jan Rabenseifner: I think lab session makes sense.

Sven Klaassen: Yes. Yeah.

Jan Rabenseifner: All right, then uh let's switch um to the first uh lecture. So um we have the introduction um and um a general introduction to double machine learning and coal machine learning and uh yeah.

Sven Klaassen: Hey, Yan. Yan, sorry. You're still sharing the the tab with the

Jan Rabenseifner: Oh uh yes thanks now it should yes now it's showing I cannot see thanks um so yes um introduction uh to um double machine learning causal machine learning and then as the second part I will also introduce the workflow uh of in our double machine learning package and maybe just before we get started I mean um from our core team I already introduced or they introduce themselves, Ven and Young.

00:13:57

Jan Rabenseifner: H but we of course also have Martin who is the co-founder of economic AI. um he's been in the field of causal inference and cause machine learning for many many years is sort of also on the forefront uh in the on the academic side and um yeah with economic AI um we have many years of also consultancy but also um training um projects um and and teaching where we try to um yeah bring the academic side towards the the industry and responsible for our trainings and uh executive education is Philip Bach. Um he's also a professor in Berlin. Um he currently is on on well deserved holiday so probably he can't join uh today. Uh but um yeah he he always monitors our training program and he was also um in the lead for the R package WML. So he did a lot of work there as well and many years in causal inference as well. Sven um introduced himself maybe uh he's always a bit humble in my opinion. Uh he's uh probably the the main architect of the Python package um ML.

00:15:13

Jan Rabenseifner: Uh so if you come across anything um any example or any class uh that you work on uh in the doublem package probably when maybe he didn't work on all of them but at least he surveyed all of them and and at some point gave his approval

that something is added. So um if you have any uh detailed questions then um he will be the right guy. And uh Jan also introduced himself. He's also our uh deep learning expert at our chair additionally. Yes. And I introduced myself already. So uh let's start with the basics of causality and causal machine learning. So in general we are trying to to merge two worlds together. uh on the one side we have machine learning uh which is um very successful in in learning complex patterns in data. I think you're all familiar with that. Um it's mostly based on correlations or more generally associations. Um a lot of it was initially introduced by statistic and math but recent developments on machine learning they also come a lot from the field of computer science and um yeah those methods have become uh very powerful at at forecasting challenges whether it's classification regression whether it's supervised unsupervised um yeah so I think we're all familiar and um now we also want to add the causal inference side um to include it um and bring it together in causal machine learning and

00:16:52

Jan Rabenseifner: um causal inference is all about learning uh causal relationships. So going beyond correlations um and uh the pioneers in that field are um Judia Pearl who um more or less gave us some or a lot of um the the language the vocabulary how to frame causal problems. Then there's also um Donald Rubin who introduced the potential outcome framework or basically he he generalized it uh for observational data that field where we mostly interested in and um then uh the latest development was uh the the Nobel Prize 2021 for Guido Ibins and Yoshua Angress and um I think that was the the the latest push it needed um to to move causal inference also into into the industry, right? And since this Nobel Prize um there was a lot of focus on this area, a lot of companies realized um how how powerful those methods are or this general cause inference framework and um yeah since then um it really kicked off and now we're trying to merge those two really on a high level um to get a machine learning um just for the basics of um uh causal modeling.

00:18:20

Jan Rabenseifner: So um we trying to look into the black box and learn something about underlying mechanism that's mostly also done in some predictive tasks that you have at hand. For example, if you have uh you want to analyze the probability that

eventually someone is developing lung cancer, um you might just use as a predictor whether um a person has a lighter in their pocket, right? Uh probably a really good predictor whether in the end you have lung cancer, but obviously that's not a cause relationship. It's not the the lighter itself that is causing uh the lung cancer, but we can still use it for prediction. Um and we want to go away from just those predictive settings and um we want to gain a causal understanding um for which is basically you can also frame it as the the real intelligence so to say and um there are a lot of example where examples where um causal machine learning um and causal inference is is currently employed for example in resource allocation um whether it's investments or you have a a new employee with a certain skill set and you want to analyze in which area on which projects you want to put that employee um where will that employee have the biggest impact.

00:19:49

Jan Rabenseifner: So uh this is also a cause a question. Then everything involving AB testing or randomized control trials normally has some uh causal question underlying. Um we also have uplift modeling. Uh we'll go through a really short example just for the notations um shortly afterwards. Uh on uplift modeling. So this is just um how to target for example with different marketing instruments you want to um nudge your customers and um yeah you want to increase sales for example that would be the uplift. Then um dynamic pricing is also a huge field um of really challenging um because of course um the pricing drives the demand and dem demand price uh drives the the prices. So you have this dynamic system and um yeah it's it's always challenging um to to find out which one is causing the other and um for example in economics you would have price elasticities as a tool. Um but um there also causal extensions to that. Then um clinical price I think um this is quite easy to see that we we have some causal questions there.

00:21:08

Jan Rabenseifner: We want to to analyze whether some medicine is um really impactful, really helping our patients or not. And um last but not least, we have also an product optimization um where we have complicated processes and we want to we have a lot of small little wheels and uh we want to analyze which um of them has the biggest impact. Uh where can we have um improve our production process the most? um with some

investments that we are doing, some production um alterations that we that we think about which is just a general overview. Then um sort of from the framing also um to to show the difference between predictive and causal modeling. On the left hand side we have the predictive modeling. Uh this is about building good prediction rules. So sort of a function f of x um x here are our features and we want to predict our um outcome y . So just introducing some notations here. Of course um in our field it's it's a lot about statistics. So you need we need some notations but notations are also helpful in the end to explain the concepts the underlying concepts.

00:22:29

Jan Rabenseifner: Um then on the other side we have the causal modeling part and this is about uh what is the causal effect of treatment B on an outcome Y . So this question looks quite simpler simple but uh of course it it already involves for example how to set up the treatment, how do we set up our outcome variable? A lot of thinking needs to go into this um this um setup and um also how to to model those causal effects there. There's um yeah a lot of different steps that we carefully need to address um where we have to check assumptions etc. And we'll see that process uh in a moment. So especially for the predictive modeling side I think nowadays gets more and more automated but the causal modeling part uh I think is is still more challenging but in my opinion also more exciting because you have to consider much more and um yeah you also need some domain expertise and in predictive modeling um yeah you have great autom automatization approaches um so yeah there's maybe also the difference between those two and um with an explicit example h if you for example have a customer churn so whether customers return or not um the question is how well can we predict this customer churn uh on the predictive modeling side and also a classic traditional question is uh which of

00:24:05

Jan Rabenseifner: the variables that I'm considering are good predictors for churn right so explainability in the Then on the other side we have um the the causal questions would be why do customer churn and um also how can we retain our customers and you can see already from the framing. Uh this is this will be much more actionable if you can answer those questions on the right hand side. um you will have an easier task in

the end um to to um sort of change the customer p pattern um the the customer churn. So this is more actionable compared to just the predictive uh setting on the the left hand side. So now also the question is uh because there was a lot of improvement on the predictive modeling part um in causal ML the question is how can we use those state-of-the-art machine learning methods for the right hand side for our causal modeling for causal inference. So how to merge those two um words then a brief introduction to causal inference. Um so um general uh the question is um how do we address causality within data?

00:25:28

Jan Rabenseifner: Uh for that our approach is normally we define our causal parameters of interest and um we need to state our necessary assumptions. So uh for identification and as well for our valid estimation uh that in the end we are interested in. And again our general question is uh what is the causal effect of treatment D on outcome Y and helpful tools for this causal inference is the potential outcome framework which was for us it is introduced by non and then generalized by Donald Rubin and uh we'll have a look at the potential outcome framework uh afterwards and then there's also a directed cyclic graphs. So this is sort of a visualization tool. Um always helps to to get some understanding of your underlying problem. We always encourage in all our projects that we have with um companies to try to draw those um graphs to get a better understanding what are potential confounders, mediators, colliders. um this just from the general causal toolkit uh that we have um and to get an understanding of which variables should be controlled for and which variables we shouldn't control for.

00:26:55

Jan Rabenseifner: So um to differentiate our our set of potential features that's always helpful um and also the D decks in the end help you to decide um which causal model works best for your specific problem. So general uh a great tool. Now um for the um pipeline so to say of uh from identification um on the one hand side versus the estimation on the other side. So we normally start with a certain causal estimate h just to to get some causal vocabulary. Here you have the causal estimate and that is um a hypothetical quantity and that's the parameter that we are really interested in right and

then in the next step um we we set up our assumptions and uh derive our stat statistical estimate. Um so the key question is here can we write our causal estimate um in as a cost quantity uh in terms of our observable data that we have at hand. And then if we can pin this down to a statistical estimate given our initial causal estimate and um our assumption set um then comes the modeling part our estimator and the question is um what algorithm will best approximate um the statistical quantity.

00:28:24

Jan Rabenseifner: uh so that we can derive our statistical estimate and the statistical estimate in the end will be the one um that we can use for business intelligence for our reports um where you can then have some some actionable insights in the end.

Sven Klaassen: just just can very shortly also to to to include if you go on back one side. I I think ju just for everybody that this is specifically so so we will talk a lot about the the specific double ML approach and the double ML package. But in causal inference sort of this this distinction is really the the the the main part one really has to consider in a lot of the problems essentially to to to try to to to get to the idea that that essentially like we we can always the later part the estimation part is to some degree right we we can always run regressions we can always compare calculate means and things like this but the the first part is really the core right of causal infence is sort called identification essentially saying like like there's always some sort of assumptions that will say we can generalize from the parts what that we actually see in the data to some sort of things what to our hypothetical quantities that that will become in the next slide I think a bit clearer but I I think the the the

00:29:53

Sven Klaassen: idea is really really important of course for causal inference to to say like I I think on ends always most In most cases, this is still the hardest part to to have find something which is plausible, right? Where once says okay that I I'm really sure that I can actually try potentially in my data see see what how effect would look like. Yeah. Sorry. Yeah.

Jan Rabenseifner: Yeah. No, thanks. Thanks. Um, yes. And also maybe for all of you just um unmute yourself if I don't see a raised hand or something like that. Just um unmute yourself if you have any question um or comment. All right. So um as as uh promised

before we have um to to introduce the potential outcome uh framework we will have a look at a specific example and that would be uh here uplift modeling. So we want to analyze the causal effect of a email campaign in this case whether or not we we send out coupons uh on our product sales or conversion that's our outcome.

00:31:01

Jan Rabenseifner: So uh and and key question normally in uplift modeling especially if it involves coupons. Uh a lot of companies are thinking about this um is uh which which part of our um newsletter subscriber should we target with our coupons? Because you normally have some customers they will have some purchases on your platform anyway. you probably don't want to send them coupons because you're potentially just losing money through sending out those coupons and you're not gaining anything. But then you have some um segment of your customers that um really um needs this nudge, right? Really needs this coupons, this additional motivation uh to make a purchase in the end. And um you want to know what what are those um customer segments uh where the coupons make most sense where they have the highest impact and that's normally the causal question. So yes and um to uh introduce some uh notation here as well for our binary treatment. It's quite simple. I think most of you will be um familiar with binary treatments. So you have um d equals to one if um the newsletter subscriber is treated that means here uh that the newsletter subscriber received the coupon and you have d equals to zero if that uh subscriber was not treated so didn't receive the coupon and now the potential outcome framework from rubins uh defines the potential outcomes with our example here y_0 would be the conversion.

00:32:46

Jan Rabenseifner: So our sales if um a newsletter subscriber receive receives a discount or our coupon and uh y_1 would be um the on the untreated side um if uh the newsletter subscriber did not receive the discount. And through those two potential outcomes uh we can construct our individual causal effect. And this would be just the difference between y_1 and y_0 . So uh ideally we want to have this effect for each of our newsletter subscribers and then we could just take the average to to know for our population what is the effect of our kos for example. Um yes but it's not that easy unfortunately. Um here comes the the quite famous uh fundamental problem of causal

inference. Uh that is our individual causal effects. They cannot be identified in general. And the reason behind that is we only observe one of the potential outcomes. That's our factual and the other outcome is unobserved and this one we normally call counterfactual. So we just have for our newsletter subscriber we only have um if someone received the the discount or coupon then we just observe by one but we are not observing by zero.

00:34:19

Jan Rabenseifner: So therefore we don't have this this difference for each individual. We can't observe that and can't h cannot in general identify this. Um so what can we do about this? Um we can take a look at other causal quantities. Um quite famously is the average individual treatment effect or mostly just um named average treatment effect. This one might be identified. Of course depending also on our problem that we have at hand and for that we normally need some additional assumptions such as selection on observable sometimes also called unconfounded assumptions. uh this one might be required and um we will try to use data to estimate so observational data to estimate those uh causal parameters. So yes for the estimation of causal effects in ideal setting maybe take a step back um in an ideal setting sort of um a lot of times called the the gold standard of causal inference we don't have to use observational data but we can simply run AB tests um also called randomized control trials RCTs or sometimes just um written as experiment and um under certain assumptions uh we can consistently estimate the average treatment effect of treatment D on the outcome Y. And this is just the um the average treatment effect is just the difference of um if everyone would have been uh treated minus the difference if everyone would have not received the discount in a potential world.

00:36:06

Jan Rabenseifner: So this would be the average treatment effect that we are interested in. And now with uh AB tests or randomized control trials, the basic idea is that we can just use our sample based estimates. So the um outcome that we observed in our treatment group um and uh the outcome in our control group because it was randomized. Um we can use this as replacements for our expected values expected values of the potential outcome y_1 and uh our expected value of the potential outcome

y0. So yes why does this work? Um in an RCT the allocation of the uh treatment of individuals is assigned randomly. I mean that's the whole point of RCTs. So the treatment assignment is independent of our potential outcomes. Uh we can see later that this is not the case for observational data. Um there we need then the unconfounders assumption where we additionally um control on our feature set on our co-variants. Um so in this setting where we have the independence of our potential outcomes and our treatment um individuals cannot self- select themselves into treatment.

00:37:28

Jan Rabenseifner: Um so we have some um newsletter subscribers who are really um all about kos and they only make purchases for kos and they um try to I don't know um sign up with multiple email address to to receive multiple kos for example. Um but in a setting where we have our RCT our AB test uh users cannot do that. They cannot self-select themselves into the treatment um because we assigned them random randomly um and because they cannot um pick the value of the treatment that is best for them in terms of our potential outcomes. We have this independence of our treatment and the potential outcomes. Um yes. And now the key question is so in an ideal world we will just run our RCTs and um we get all the effects we are interested in but a lot of times um through um financial constraints legal constraints constraints uh we cannot runs in all settings so the question is now um how can we evaluate causal effects if we don't have any randomization So we are interested in causal effects from observational data and for that we normally use this um selection on observables our conditional exogeneity or underlying unconfoundedness assumption.

00:39:00

Jan Rabenseifner: Um yes where we basically say that without randomization treatment and uh potential outcomes they might be related. So we have some confounding factors. They are displayed here on the right hand side right in this graph where we have our X those are our confounding variables and they impact both our treatment D as well as our outcome Y. And um now we have this confounding. So we know that our um treatment assignment is is not at random anymore. It's it's caused by some underlying factors. Um but then we can add our uh key assumption and that is a conditional on our

observed coaritantss and our treatment is as good as random. So um if we have a rich set of features and we condition on them then we gain back our property that our treatment is independent of our potential outcomes. But for that we need to control for our uh confounding um variables X um here in the graph in the in the blue dot and interested of course if we also I think there's a question

Ani Pole: Yes.

00:40:19

Ani Pole: Um I just have a question.

Jan Rabenseifner: Yes.

Ani Pole: So you when you think about linear regression, right? Like you put all the variables there and um you get partial correlation of one variable with the target variable that you're trying to predict. And in this case, for example, if you were to run like a simple simpler simple linear regression of um X and D on Y, even if there was kind of like non uh it wasn't random the assignment of the treatment, why couldn't that be considered as um kind of like average treatment effect?

Jan Rabenseifner: M um you would need more stronger assumptions that this is a causal effect as well. Um but maybe on the theoretical part you also have Sven as an expert. Maybe you want to comment on

Sven Klaassen: It's so so and as we see later so linear regression can be can be used to to estimate causal

Jan Rabenseifner: that.

Sven Klaassen: effects. So, so essentially like you said it's um but I I think what you meant right that is essentially um the the interpretation whether it's causer right that that needs the the other assumptions right so so essential you can run the linear regression right and and you you put in um like if you have like um variables x but you also have z and um w and things like this you you can run a regression And you can add W, you can add Z,

00:41:47

Sven Klaassen: you can add X and your your coefficient might change for your treatment, right? For if you add adding them in partial correlation and the question is which of them has your sort of your causal interpretation, right? And and and the so so

conditioning on X so so including the is is including it in the linear regression, right? But but essential still so and then it has this causal interpretation. So I think that that is fine. But the so the interpretation comes really from the idea um what are the other features I have in in my problem. I am conditioning on the right variables. So so am I comparing the right units to to each other? So, so essentially in linear regression you're comparing then units which have similar features X and just varying then the treatment right you're saying like like essentially you're saying okay if all the X's I'm including in principle it compares then units like for example XS H similar in similar age whether the effect change is is is um yeah constant if it's just added but if you have interactions you you can also have like like like the the these interactions effects but you you essentially compare the units at different levels of age for example and other things.

00:43:08

Sven Klaassen: So, so, so, so you can make it cause causal for this. Is it but but really there's really this distinction like like the linear part is the estimation and and whe what you put in as X is sort of the identification part for for in in this line linear setup and but one caveat right for linear regression you're making another really strong assumption and it's linearity right you you you having the additive model that every part of X and D is sort of an additive linear component component right you have $\beta_1 x_1 + \alpha_d$ and things like this so this is strong functional form assumption additionally to

Ani Pole: Okay,

Sven Klaassen: this

Ani Pole: thanks.

Jan Rabenseifner: Yes. Um maybe to add that if you have high dimensional problems, linear regression potentially you have multilinearity etc. um is not working too well then you want to introduce some shrinkage and we'll cover that later that this introduces some bias as well. So if you add some penalization because you have a rich set of features um then you will have a direct bias and then we we need more advanced tools for that that we will cover later as well.

00:44:27

Jan Rabenseifner: Thanks for your question. All right. So um yeah uh the intuition behind the controlling for our um confounders on our um X is that um if we compare our

individuals with the same underlying characteristic X through conditioning on our covariates. Any difference that we then observe in our outcomes can be really attributed to the treatments. So we're back to an apples to apples comparison and the methods to enable this this would be uh regression adjustment uh where we uh model our conditional expectation of our outcome uh based on B and X and we average over our covariates. Then there's also rewriting approaches uh where we weight our observations by inverse propensity scores. In propensity scores are just the probability of being treated. So you run for example a classification model uh binary classification whether or not someone is treated with our sets of features. So you're basically modeling uh this part disconnection from the blue dot towards the orange dot and uh you create your propensity score and then you uh reweight those observations that uh they become more similar towards each other and then there's also the doubly robust methods and they will be introduced in more detail later and that's where we combine both approaches.

00:46:04

Jan Rabenseifner: So we have a regression adjustment as well as retraction. Now uh let's get to double machine learning.

Sven Klaassen: Yeah.

Jan Rabenseifner: H yeah,

Sven Klaassen: So,

Jan Rabenseifner: you want to add

Sven Klaassen: so yeah. So, just just to come back to the question. So,

Jan Rabenseifner: something?

Sven Klaassen: regression adjustment that that you can have slightly different forms, but essentially like linear regression would be just fall under the type of regression adjustment. You're just modeling the conditional expectation in a linear form in in in this part. So, you're just running linear regression to to to predict this in a linear form. So it's a pretty valid point on this.

Jan Rabenseifner: Yes.

Sven Klaassen: So yeah.

Jan Rabenseifner: Thanks. Uh, there's one more question. Yes.

Chieri Naito: um not a question but I was thinking about Annie's um question and I think Sven um answered u you know uh but uh from my project I was also thinking the same and I think the difficulty like for example as learner is the most basic you know thing we can think of to um get this treatment effect but the thing is that what we want

to know is the you know the how much the change in t changes y this like pure effect.

00:47:21

Chieri Naito: But if we include uh every every every like features uh you know x and uh and instrument variable and everything then you know these not even if you know it's a machine learning model not a linear model uh there's a regularization effect so the effect to um you know the effect becomes smaller and smaller and so um it becomes harder to uh like measure this treatment effect uh correctly. So I think u that's why um we need a double machine learning.

Sven Klaassen: Uh yes. Uh exactly to to some degree like like what Jan said,

Jan Rabenseifner: Yes.

Sven Klaassen: if you include more and more features and you then add some sort of regularization like ridge penalties or things like this,

Chieri Naito: Yes.

Sven Klaassen: then then then you of course your coefficients get get shrunken, right? And then then then you you have to readjust them a bit to to to to find or to to for for the bias that's introduced by your your regularization methods. Yeah, it's true. Thanks.

Jan Rabenseifner: Have you already tested some double machine learning methods so far?

00:48:36

Jan Rabenseifner: What kind of modeling approaches have you additionally tested?
Just out of

Chieri Naito: Yes. So we had a price elasticity project.

Jan Rabenseifner: curiosity.

Chieri Naito: So try to um measure uh the effect of a price on the conversion rate and uh we tried almost from scratch. So everything so S learner, T- learner, um uh DR learner uh which my colleague tried and then we moved on to double machine learning and uh what we tried is our learner and uh uh coal forest and the cause of forest performs the best. Uh so what we wanted to uh measure is not eight but Kate so conditional average treatment effects. Um yeah, but uh

Jan Rabenseifner: Yes. Yes. Uh probably the right direction to go, but uh yeah, maybe we can discuss it during the lab or something like that in more detail. Thanks for the All

right then. Um yes. So uh introduction here comes our field that we are excited about this uh double machine learning. Uh so um why do we want to use machine learning for causal inference?

00:49:47

Jan Rabenseifner: I think we discussed it briefly. The machine learning methods or algorithms are powerful tools for predictions. Um as a use case continued here we have our uplift modeling. we might want to run a experiment that would be um AB test um to improve our precision efficiency and power. Um if we can't run an experiment uh we have to rely on observational data there we need to account for confounding and um we also have a highdimensional vector of coariantss normally um depending on how well we um how well our data set is on our customers um and we have complex relationships uh of y and uh d as well as um x on d um and x on y . So basically all those um relationships between our different um variables that we have um they are usually uh quite complex and um machine learning can help us to model those relationships um going beyond for example linear regression. So yes, one more

Tsz Yan Lam: Can I ask a question question here? So I think typically in the more correlated

00:51:06

Jan Rabenseifner: question.

Tsz Yan Lam: or predictive version of machine learning I think many people have the tendency to just basically throw everything in it in terms of the features. I believe this is probably not a best idea for crucial machine learning because you can potentially open up other path the backd door path. They is this something that we would uh mention when we are doing the work uh the uh the notebook section or something that we can we we would talk Oh.

Jan Rabenseifner: Yes, I think especially the first notebook session focuses on on that part as well on which uh variables we should um control for and where we also have a downgrade of our performance uh if we control for variables where we shouldn't control for. So that will be a part of it. All right. Yes. So um now we are getting in the direction of uh the the estimation side our estimator what algorithm to choose uh to get from our statistical estimate that we set up um to our statistical estimate and um double

machine learning can help with that.

00:52:31

Jan Rabenseifner: That's that's one possibility. Um of course as as we just hear there heard there are also alternatives around but I think the um double machine learning framework is quite flexible espec es especially our um package implementation has um this is constantly growing and has a lot of features. So hopefully there's an algorithm um included that can help for your specific uh problem that you have and that you're considering. So um yeah it's sort of our double machine learning merges um to two separate sides or that have been separated. Traditionally we have um our prediction methods uh where we regarding Python we have the psychic learn package. I think it was uh from a French research team who introduced psychic learn. Um yeah and this is one of the the biggest projects probably on on on GitHub that you can find open source projects. Um then we also have um other um machine learning approaches uh such as lasso um sort of this um one penalization then we have random forest uh we have boost to trees whether it's flight GBM X boost XG boost cat boost um yeah we we have a lot of methods uh to consider on this side it's quite advanced field and then we have this this uh really old toolbox uh from econometrics and statistics where we have structural equation models.

00:54:07

Jan Rabenseifner: Uh we have our tools for identification um our possible assumptions that we could consider. We have um asmtotic prop properties. We have our hypothes hypothesis tests confidence intervals. So in the end um all those tools for valid inference on the right hand side and we want to combine them in causal double machine learning uh such that we have really high predictive power but at the same time valid inference and we sort of extract um some parts from both.

Daniel Jacob: Sorry,

Jan Rabenseifner: Yes.

Daniel Jacob: one one question regarding the machine learning methods.

Jan Rabenseifner: Yes. Sure.

Daniel Jacob: So I know that in the um original machine learning paper I think Jukov they tried a variety of them and showed that um you can reach root n um consistency or

asymptotics. Um and then I think later on there was also a paper showing this for neuronet networks. But do you guys know is there a limitation like are there machine learning methods that you also tried where we cannot kind of prove um um root end conversions or

00:55:18

Jan Rabenseifner: Um, so I don't know. Sven, do you want to take this one

Sven Klaassen: Yes. Yes. Um so so uh like I I think to to to prove right

Jan Rabenseifner: or

Sven Klaassen: the the convergence rate essentially um is usually done right under under very strict assumptions to to to get the the actual convergence rates right so so for example if you have lasso or linear models you you have like sparsity assumptions to say in the these high dimensional settings or if you have neural networks you have like the standard feed forward parts and so just feed forward structures and things like because you need like for the theoretical proofs really really strong things and also for boosting and and random forest and things like this but essentially what what is in in the the general framework what works and and we'll come to the the the um the the properties uh quite uh soon is essentially you you need just uh predictive power right at um like like at the fourth root generally in in this so so it's it's a

00:56:27

Daniel Jacob: Oh, let's

Sven Klaassen: bit less but but I think what what matters then in practice is essentially that you get good predictive power right you you you never know what whether you in your example the predictive power is sufficiently well right but but then the the

Daniel Jacob: look.

Sven Klaassen: the I think the practical approach or what we are doing is essentially to try to get the best predictive power you can and then you have a ch then then then you have sort of the the that that should be the best approach that it works like and and then so so um what we are using quite often are essentially like linear models as baseline then some boosting models and maybe currently we are trying to to all often maybe use some sort of the new um tabular foundation models things like this like top PFN and things like this because like if they good are very good at predictive power right that

that's a good thing about this framework. Essentially, it's it just comes down to the predictive power of your your your plug-in machine learning methods.

00:57:28

Sven Klaassen: Then then it's the rest is essentially um you don't need so the predictive power is the thing that matters for the asymptotics of your your your point estimate for

Daniel Jacob: Yeah,

Sven Klaassen: this.

Daniel Jacob: thanks for elaborating on this one.

Jan Rabenseifner: Yes. And um also if you probably it's not so much the case inside um your company but if you have smaller data sets you need to be more careful also with boosting methods uh because you get overconfident estimates towards the the borders especially in the propensity score estimation close to zero and one and this will give you extreme weights. So um that's also a part if you uh don't have a large data set at hand that you should consider with them. can be a caveat there. All right. So, um we want to merge um those two parts um to have um predictive power as well as uh valid inference. And now the question is uh why don't we just uh plug in our machine learning predictions. So in a naive approach, we use our machine learning um nuisance functions we make our estimates and the nuisance functions that's also terminology used in our package a lot.

00:58:44

Jan Rabenseifner: Um nuisance functions are just um the outcome regression. So this is this conditional expectation uh of our outcome conditional on our features and um on the other side our propensity score estimates. So the probability of treatment given our covariates and why don't we just plug those predictions um into uh the estimation of our treatment effect θ . So um key intuition here is machine learning optimizes for prediction accuracy but not for unbiased causal estimation. So this is uh the the key crucial difference. So the problem here is that our machine learning methods they they use regularization whether it's some penalization early stopping or shrinkage methods right um and this uh regularization introduces regularization bias in our nuisance estimates and uh this bias also propagates then uh to our causal estimate that we are interested in. So the naive approach is not working well. Um so we need a framework

that uh immunizes um our cause and estimate against those uh regularization bias that we observe. So uh small errors in our nuisance estimates they can cause large errors.

01:00:08

Jan Rabenseifner: We um can see uh on on the right hand side the problem with our naive approach is that uh the estimation error that we have in our nuisance functions uh creates also a first order bias in our uh causal estimate. So um therefore it can be uh that even small errors in our machine learning predictions they will have a substantial bias in our causal estimate and if we have a substantial bias then our standard inference will be not valid right so we can't use our uh confidence intervals reliably we can't use p values um all the underlying inference then um yes becomes invalid and so that's that's the crit critical part here and um general setup that we have is uh we want to estimate our co parameter. We have our nuisance functions and our machine learning methods. They will give our estimates for the uh nuisance components. But the question is now how to use them to not to not have this first order bias to not run into this risk there basically. And now comes the the first ingredient of double machine learning that is neon orthogonality.

01:01:28

Jan Rabenseifner: We try to uh create those orthogonal scores. And um the idea is that we construct our score functions such that um the errors that we do in our nuisance functions uh have only second order effects on our causal estimates. So basically you have small errors um from our different nuisance components and a product of those small errors then has a negligible impact on our overall causal estimate. So um our aim is that uh we have um or we know that we have our regularization bias uh but we want to make sure that it doesn't infect our causal estimate that we are interested in. So we can still have valid inference. Our confidence intervals for example have the correct coverage. So that's um why we introduced this name and orthogonality. Um we also try to um I mean you you can sort of explain this name and orthogonality in a more mathematical sense. We try to keep it here on a on a high level. Uh we also want to add some intuition. So you can think of this uh score function being set up that you have um in a grid sort of two directions.

01:02:48

Jan Rabenseifner: So you have your horizontal axis for example um would be uh your nuisance estimate and your vertical axis would be your thetas or your directions that you have in the space and you set them up that they are orthogonal to each other. So perpendicular and um that means if you from our true estimate um you make a small estimation errors through a bias that you have through your regularization. If you make a small error h in your um nuisance direction it doesn't change the position of your um causal parameter your theta direction. So uh small errors there uh don't influence your um theta direction your causal estimate that we are interested in. That's just I hope this is helpful a bit uh the geometric intuition there and good news for you is uh double ML or ML package automatically uses orthogonal scores. That's one of the um key components that's always included in our different classes that we have. So you don't need to do the the hard work to derive them. Um but of course if you want to extend our package, we always encourage some uh extensions uh then you need to really if you want to add another class um you really need to think about how to set up those organ scores if you have a new model that you're interested in. So um and there comes the the scientific sides academia into play where a lot of people are trying to set up uh new models uh with using orthogonal scores.

01:04:34

Jan Rabenseifner: Then um the second ingredient uh that's uh was was uh Daniel's question or the direction of Daniel's question is also um that we need high quality uh machine learning estimation. So our nuisance functions must be still estimated well enough. Um so uh orthogonality just reduces bias from our nuisance errors but just to some degree. Um our errors in our nuisance components can't be arbitrarily large. Um so uh the product of our uh machine learning errors from our different nuisance components that we have. So outcome regression and propensity score estimation. this product uh must shrink still fast enough. So um that's also part of the Victor Jenosukov papers and and subsequent sequent papers that followed. Um what this allows um we still have that machine learning methods can converge slower than classical parametric rates. Um so just an improvement on that side uh and it enables the use of uh whether it's random forest brad boosting neural nets um all of this rich toolkit uh from uh the

machine learning uh community. So practical guidance here uh if you have a large data set still use um flexible learners that are appropriate for you.

01:06:04

Jan Rabenseifner: But um that's probably one of the more recent steps that we sort of always suggest now is um also do a um proper hyperparameter tuning for your um nuisance functions that you're estimating. Um yeah there's also a paper by um some colleagues of mine that showed that um the impact of hyperparameter tuning hyperparameter tuning is quite large and we will also have another session on hyperparameter tuning um after this session. So we will dive uh into that in in more detail. And then uh what's always helpful as well if you consider multiple models for your nuisance components is of course also uh to evaluate the prediction quality. So some out of sample metrics uh that's always included in our double ML workflow as well. Uh so you can just uh in our package calls uh in the summary print uh you can already spot the prediction quality and if you test multiple models um you can just directly compare them. Um and as a rule of thumb if your underlying models um are poor uh your causal estimate will suffer. So um yeah also spend enough time although a lot of uh those parts is nowadays uh automatized um I still spend a lot of um time on the um nuisance prediction modeling uh this always helpful for the causal estimate as well.

01:07:38

Jan Rabenseifner: Then um our third ingredient is um sample splitting. So we want to prevent that um overfitting that we potentially do in our um through our machine learning models in our training data that it also contaminates our causal estimates. Um so if you just simply use our same data for training our machine learning um nuisance components as well as for the causal estimation. This would create some bias. we would have data leakage um eventually. So uh we want to make sure that our overfitting errors in the end um are not correlated with our estimation errors and if we are not doing sample splitting then we're risking that they um become correlated with each other. Now the solution that we have implemented is uh crossfitting. I guess many of you will be familiar with cross fitting. It's quite simple concept. So we split our data into k folds. Um bf fold here has five. And then uh for each fold we train our machine

learning methods on the other folds. So for example on the other four the first four folds um we train our model and on fold five um we then make our held out fold predictions and then we swap the roles.

01:09:05

Jan Rabenseifner: That's the the crossfitting component. Um we swap those roles. So for example then uh we take u the first three folds and fold number five to train and uh predict on fold four and we we sort of um run this through till every fold was used also uh for the prediction and then we just collect our predictions. So we have for each observation uh was used for for training four times but also um each of them uh was used for predictions. So we just collect those predictions that we have for each observation and we h construct our scores and um benefits of this approach are um first of all we are still we we ensuring that predictions are always out of sample. I think that's that's always key and um additionally we have that all data is used for both training and estimation. So it's more efficient um than just using a simple train test split, right? Because we use the data for for everything and we're not losing any data. Um so we're not restricting ourselves there.

01:10:19

Jan Rabenseifner: It's more efficient. Then uh comes the main result of double machine learning. Uh if we use our key ingredients of double machine learning, those three that I just introduced. So name of high quality machine learning estimation as well as sample splitting. Then uh this is from the famous paper of uh Victor and co-authors from 2018. Um it states that under some uh also might regulatory uh conditions we have that uh our uh doublem estimator is consistent and um that our um sampling error is approximately also normally distributed. And then also part of that paper is how to set up this this variance term in a general fashion that can be extended for um multiple classes that we in the end implemented in our demo machine learning package. Um so that we also can have um valid inference construct our standard errors and uh confidence intervals. So that's also part of of this uh famous paper. Um yes then uh getting started with double machine learning. So um for the uh implementation uh of our um of this uh theoretical framework in our ML package the key ingredients are of course setting up those orthogonal scores.

01:11:54

Jan Rabenseifner: Um then in general the package um Sven has more more details probably on that but it's a objectoriented implementation. So we have um multiple components that we'll see later where you for example set up your data you set up your model um your your nuisance function um and you can um because it's object- oriented you can also combine um whatever suits best for your application. So it's quite flexible in that sense and this object orientation also helps sort of you you have this this template how it should be constructed. Um this helps also in um when you want to add some extensions to our package. Um then you have sort of a a standard workflow on how to go through set up your scores etc. Uh so this object orientation um helps in the growth of our package and um we always um cheer if if someone wants to contribute to our package. So um yeah if if you have any ideas you you had some research project that maybe fits into this model and um yeah we we can always try to help with that process as well.

01:13:09

Jan Rabenseifner: Um yes and this um exploits a common structure that center a road around those um score functions. Um then our package is also uh specifically built that you can use for this nuisance components or outcome regression propensity score learner uh that you can use state-of-the-art um machine learning uh predictions and tuning methods. Um we just when we go through the workflow you can see that uh this can be quite quite easily achieved. Some of you have already experimented with our package. So um I hope this wasn't an issue or you can easily use all the machine learning prediction methods that you you're familiar with and uh to achieve that we um sort of focus on the same API as psychic learn. So psychic learn all the methods from psychic learn are enabled in our package and also because psychic learn is so popular a lot of the other machine learning uh packages um they also have a similar API to psychic learn. So all of those similar learners um can also be used in our package as well. And then uh we also have a general implementation of sample splitting.

01:14:25

Jan Rabenseifner: There are also some options on how to alter or manually set how you want the sample splitting to be done. Um but just the default is this uh five-fold cross fitting normally. Yes. Then um as an overview of our uh current implementation um or our current models that are implemented in our package. So uh this is always constant work in progress. Uh since um last year probably there was an addition and uh if we give the same presentation next year hopefully there will be more models listed. Um so this is uh always work in progress and we always have some PhD students contributing not only from our chair but also from other chairs. Um so yes we we we're trying to create a framework that can be used with um um yeah at least the most common questions uh that you have in in causal inference or causal machine learning um and also some some more um in in certain niches uh where you have like specific questions at hand. we we are trying to provide the the models for that and um the first two um they were already introduced or adopted from the uh Victor's paper from 2018 is this um general class of partially linear models.

01:15:50

Jan Rabenseifner: Then there's also this uh nonparametric uh interactive regression model. Um for the partially linear we also have continuous treatment um possible as well as binary treatment. Um both of them has as underlying data structure cross-sectional data and we have uh different causal estimates also uh that we um can sort of estimate using those those model types. So it would be the average treatment effect. We also have for heterogeneous treatment effects. Uh we heard that earlier from from your project that you also worked on Kates. Um so conditional average treatment effect and group average treatment effects they're also enabled in those model classes. Um for interactive regression models you also have the average treatment effect on treated and um then an extension of the interactive uh regression model is uh the potential outcome models. They additionally allow for uh discrete treatments or categorical treatments as well. And uh here we also have um uh the option for heterogeneous treatment effects again with this conditional and and group variants. Um probably a lot of you heard already from instrumental variables.

01:17:10

Jan Rabenseifner: That's also a quite quite popular direction. Um especially if the unconfoundedness assumption doesn't hold um a lot of times you try to construct some IVs. Um and here we also enabled uh continuous as well as binary treatment and um as a causal estimate you have the average treatment effect and also a local average treatment effect. Um then quite recently we also added a difference in differences. Um so it's a merger of of two very uh popular approaches. Difference in differences was used in economics for probably the last 10 years. It was probably one of the most popular tools and um through some recent uh research we also constructed difference in differences with non-orthogonal scores. So a merger of difference in differences and double machine learning as well. And this one also has a binary treatment variable enabled. And um yeah data structure here is then panel data or repeated cross-section. And the treatment effect that we had our causal estimate is the average treatment effect on the treated. Then yes there's a

01:18:27

Tsz Yan Lam: Yes, sorry.

Jan Rabenseifner: question.

Tsz Yan Lam: Uh I think maybe you can continue but I think one question that I really have is so you have a treatment column here to say what kind of treatment is that particular model type allow. How about the uh outcome whether if it's continuous or

Jan Rabenseifner: Mhm.

Tsz Yan Lam: binary? Do you have a restriction on that or or there's no restriction?

Jan Rabenseifner: H great question. I think um because Sven has the best overview about our package, I will pass this question on to Sven.

Sven Klaassen: Um um yeah. Okay. Um so yeah, maybe we we should add add an an an allowed outcome column to it. Essentially that varies a bit, right? Most of them. So so so I think generally most of them should be um applicable to either binary or continuous types of outcomes in this part. But it's in all of the settings, it's assumed to to have like a single outcome. So not multivariate outcome. Else you would run sort of multiple estimators um multiple models essentially.

01:19:33

Sven Klaassen: And and um so so sometimes it it changes a bit. For example, for the partially linear model, it's a linear model. So so a binary outcome would imply sort of an additive probability. So therefore there's a logistic version of the partially linear model implemented in in the current version. So so I think most of them should work essentially at least the the three four like the outcome is generally like continuous or binary um should should be possible from from most nearly all of

Tsz Yan Lam: Thank you.

Sven Klaassen: them.

Tsz Yan Lam: Um,

Martin Bierey: No, go ahead.

Tsz Yan Lam: one thing that I would like to take into deeper in the in the in the binary case because I remember when I try to learn it myself, one of the thing that are really being done in the double ML is looking at the residual and residual relationship and for the case of continuous I think residual is quite well defined but for the case of binary because the target itself is one zero and even if you're using a probability as the model prediction.

01:20:44

Tsz Yan Lam: Do you then consider the subtraction of the one zero to the probability as

Sven Klaassen: Um

Tsz Yan Lam: the residual in that sense or how are you doing it? because that is the problem that I I don't know how to resolve and I I and would like to hear how uh how do you think

Sven Klaassen: yeah and it's that's a very good point. So so essentially um the the um the the residual on residual regression from this part comes from is the intuition from the orth orthogonality assumption in the partially linear regression model. So in the partially linear regression model because there's this additive structure of the treatment effect you you you have the the residual is essentially um you you can I I may maybe some of you know the first wall level theorem like you can partial out in a linear model. You can remove um factors one by one by running so sort of one one regression on everything else first and then on the residuals running a second regression and then you still get the same coefficient if you run a large regression at the first step and so sort of this idea just works in the partially linear model and it really relies on so this residual idea on this additive structure right and and the additive structure for the the the binary

outcome model doesn't really make a lot of sense because it is like this additive probability model.

01:22:13

Sven Klaassen: So it would say like at each point you you get an you you increase the probability by 0.5% right and 0.05 like like or some something like this. So so I I think you you can run it and then this residuals are defined but I think for most use cases this additive probability model is is not really really applicable. So therefore in these cases I would say like a um either like having this completely interactive model or um a logistic version of the model is more appropriate and then you don't get the residual on residual regression part then this changes because you don't have this additive structure that this changes to the sort of um the the idea that that you sort the the plug-in part so your machine learning you you have a score all of these models and that's the underlying concept Right? We on this very high level all of these there are scores available. You estimate some part of the outcome, some conditional mean of the outcome, some propensity score and you plug it in a certain type of score and essentially the score is has this property that that that's is is somewhat not that much affected by small mistakes or small errors you're making in in these predictions.

01:23:29

Sven Klaassen: And uh so this sort of robustness property or orthogonality property and um yeah and in the special case of this partially linear model that's residuals and and and there therefore I I I can see the point that it doesn't make a lot of sense to to to the the the binary outcome but but that is because because it's a very spec specific interpretation essentially of of the general idea is this also gonology assumption

Jan Rabenseifner: Yes, I think there was one more question by

Tsz Yan Lam: Thank

Martin Bierey: Um yes I have one question.

Jan Rabenseifner: Martin.

Martin Bierey: So on the model types that support the group average treatment effects um are these cases where you need to predefine the groups or is there also support to basically form

Sven Klaassen: the

Martin Bierey: groups where the package would find that the treatment effects most strongly differ?

Sven Klaassen: generally predefined groups. I some sometime there's something on the interactive where you can f essentially try to find some groups with the so with a sort of tree based approach but but most of them are predefined groups but from our practical part essentially people already have some idea what what are sort of key groups they they want actually they want to intervene on and that that are also relevant right and and what what happens Often when you try to really optimize this

01:24:59

Sven Klaassen: automatically, you have to be very careful that you don't put in too many features because you're essentially overfitting at some point and you're fitting a lot of noise to to to just um try to target treatments very specifically in very small groups that because uh you you have to have to average things out. But may yeah maybe we we have in the later in the day a session on on heterogeneous treatment effects. So so we will also see look at groups maybe I can I can say something more on these ones but essentially most of them are saying like you said predefined groups. Yes.

Jan Rabenseifner: Are there any more questions? Then uh yes there um after difference uh there was also a sample selection added. Uh sample selection is uh a model um if you have some um outcome observations uh missing. So whether it's at random or not. Uh but it's also enables a model where you have um some missingness in in your data. Uh which we um from uh a lot of practitioner heard uh is a standard problem right um so um this we we want to add that and that's

01:25:57

Sven Klaassen: Um

Jan Rabenseifner: also one of the extension extension of our classes that was already implemented. Uh and then we also have a regression discontinuity. So you have some cutoff event uh where I don't know everyone slightly below didn't receive the treatment, everyone slightly above a certain value uh receives the treatment. Um in those settings um you can estimate the the local average treatment effect because units around this cutoff are quite similar towards each other and uh this was also one of one of the extensions we added. And then um there's also a quantile treatment effects uh enabled

as similar to that or the adaptation was also the conditional value at risk. Um and we have rated treatment effects um also um multiple treatments also enabled. We see that in the example later. Um yes. So the the list of extensions uh is also constantly growing. Right.

Sven Klaassen: great I I think from from a time perspective it's a good point right Yan to to wouldn't to make

Jan Rabenseifner: Yes,

Sven Klaassen: a break right now right uh and and then because we look so the

01:27:33

Jan Rabenseifner: I Yes.

Sven Klaassen: time table is more or less like a general overview the hyperp tuning part is a bit shorter so so I think fits still very good in in in time on this point so so I would Hey, we're doing the the the short break now and meet again like 5 11 if that's fine, right? Um no, like 10 minutes. Great.

Jan Rabenseifner: What?

Sven Klaassen: And then then we can continue with the then then we will see actually how how things are estimated in in this part. So the current one is was very high level and specifically just getting the very basic concepts and we try to to match it then to to the application.

Jan Rabenseifner: Yes. Right.

Sven Klaassen: Okay.

Jan Rabenseifner: Thank you. Then enjoy your coffee break.

Tsz Yan Lam: All right. Thank you.

Sven Klaassen: Okay, then I think right we we I hope we we can continue. So I will just at at the start so that everybody has time to go back. Just a very short um oh sorry one uh size.

01:39:36

Sven Klaassen: Okay. So, so we will now go through the the the basic workflow and just just to recap essentially the basic idea is at first or why we had the the first section is to just have the the general ideas of causality essentially and specifically what are the ingredients or the specific part about double machine learning and these are like like

presented these three type of ingredients and in the workflow you you can essentially see how these or hopefully We will discuss how these specifically relate to certain steps there and um or or how how these generally relate to these steps. And the main idea was still what we had before and you will see this in the notebook that was already asked a bit that that double ML is not a guarantee to write to get causal effects essentially. You still have to think about what you put into your model what essentially your model is essent is estimating. But that is true for all causal inference methods like even for cause of forest and things like this you have the exact same same problem exact same same issues for this.

01:40:43

Sven Klaassen: Okay. So so how does a typical workflow look like before we go to the the tuning essentially I I think or just to get the high level ideas of course the steps don't have to be in the the sort of exact same order but of course at first you you should take a look at the problem formulation. So nothing double ML specific. So it's sort of trying to see what is the data actually I I have and which effect I'm actually estimating and therefore just drawing or visualizing a short dependence of my features my available features my maybe not observable features in in a deck is usually helpful to to get an interpretation or a feeling specifically of the of the problem. And um in in this example what we will be here it's a just I think you will go through the steps in more detail in the notebooks but essentially in this example the goal is to estimate the some average effect of um being eligible on on the 4001k pension plans right and so sort of the and the the the being eligible that's called of an intent to treat effect because of course the effect is you you only get the effect if you participate in this plan actually right only then you are affected but whether you participate in in the plan in these examples sometimes depends on some unobserved characteristics for example saving preferences things like this those different type of other behaviors and the

01:42:21

Sven Klaassen: the only thing you can actually intervene or more or less is something like the eligibility whether you actually are have the possibility to participate but whether you participate or not that's still up to the individual right whether they want to choose to to participate in these type of saving plans or not. So, so sometimes these intent to

treat effects like whether you have the option to do something that is more more useful in these examples and the idea is just so so in this because this is not the use case we want to discuss in detail is essentially we have a treatment which is a binary whether you're eligible or not we have some measured covariates X essentially which affect whether you are eligible right it's only specific on companies and they also affect your outcome it's net the net financial assets in this part. And so so we we already heard this a bit, right? That the idea is we want to isolate the effect from here, the overall effect on the net financial assets, but we want to condition on the other features which also affect both of these to really isolate only this type um this type of sort of correlation essentially to give it a causal interpretation.

01:43:36

Sven Klaassen: That is just the basic idea. with the deck we will go through in the the example a bit more detailed comment types. So, so how it is done in double ML? Um, essentially at first like we said we have to define everything specifically what we want to estimate and therefore we are using sort of data objects which is essentially sort of either pandas data frame back end where you have to just set up certain columns or specify certain columns which are used as outcome. So that's a Y column, right? Then what what is used as treatment and what are my conditioning variables there? We we have a general set is X and there you see it's age, income, education, things like this which we want to condition for and um these this is a very basic structure and depending on the class there might be a bit more. For example, if you have difference of differences you have to set up what is sort of the unit identifier. So, so because you observe units over certain time periods and things like this, but at first you have to sort of specify in your data what what you actually observe and so then maybe not in the same same order but essentially the the idea is maybe you have to choose a certain model and specifically what what I mean with choose a certain model is in this part you have to think about what are essentially estimates or estimands I would like to know in my data.

01:45:03

Sven Klaassen: and what I can sort of identify from from the data in these examples. So so Jan already had went through the list what is actually sort of on general types is

implemented in the package. So all of these models are um are implemented to to um have this specifically orthogonal score functions. So we said like one ingredient to apply this double machine learning procedure was the orthogonal score function and for all of these problems there are known score functions which have these type of orthogonality properties. So they fit essentially in this framework. So so this is the first thing and the question is then specifically in this model do I want to have an average effect? Do I want to have a conditional effect essentially? Then is is my treatment binary or not? Or maybe I think like if if one has something like wants to do a difference a difference study over panel data essentially do do I want to use this type of difference and difference procedure. And um the second ingredient was that we use machine learning methods which are somewhat well enough.

01:46:18

Sven Klaassen: Right? So, so we want to set up our machine on flexible learners. We want to maybe um say okay we want to compare whether that makes a difference if we run a linear regression for a problem or we estimate all these effects in a non-parametric way essentially um by running boosting algorithm or random forest things like this and therefore one has to set up um or one has to define which machine learning methods do do I want to use um and you like we that you can use anything which which uses the scikitlearn style API because we want to have this fit predict methods. We want to have clone the estimators essentially to to to have several cross fitted versions. So essentially all all of these which have a scikitlearn style API they are viable you can use them and this can be a linear regression that can be something like xg boost and you you can set them also with all these these um so sorry my mouse is working a bit strange um but but um you can set them all the hyperparameters you you want to use but essentially you just have to choose different learners and which learners you have to choose depends a bit on on the model essentially.

01:47:39

Sven Klaassen: Um the and the the you have to choose normally what we have you have to choose one model for the outcome and one model for the treatment. So, so you want to predict if we go back for example in the this interactive regression problem you want to have one model to predict the conditional expectation of the outcome and one model to

predict the conditional expectation of the treatment but because the treatment is binary the conditional expectation is a probability so it's a propensity score right so so you and um so so um the the um so you need a classifier for the treatment If the treatment is binary usually and you need a um regression problem for for your outcome if it's continuous of course if your outcome is not continuous sometimes you also need a a classification classifier for your for your outcome right really usually they are called G and M. So G for your outcome regression, M for your treatment and depending on what type they are, you need is usually um like I said a regression or a classific classifier and in this case what we will be choosing um for this one is I I think it's is the interactive regression model and therefore we will set up for our out uh no the partially linear model I think for for our outcome we set up um or let me see I I didn't know the interactive regression model.

01:49:13

Sven Klaassen: So for our outcome I don't know the naming is a bit off we will choose one regression just with Y XG boost in this very simple example just setting up some XG boost regressor and for the treatment because it's binary the eligibility part just 01 we will choose just the regressor to plug it in. And yeah, this question

Tsz Yan Lam: Yes. Can I ask a question about what is the requirement for the psychic learn like uh uh model? Is it like actually if I have something some model that don't have the psychic learn interface I can also wrap it around? So as long as I have this feed and predict function or there's some other requirement.

Sven Klaassen: Yes. So, so, so, so Psychit Learn has essentially the the the options to to create new learners if you just implement these fit and predict parts yourself. So, that is possible. Um so so like this usually the standard methods all have this type of API like like I said like light like GBMXG boost top PFN things like this but um you can also in in a lot of models you can also just use external predictions if you would like.

01:50:20

Sven Klaassen: So, so in in some sense depending on if if things get somewhat complicated, you can try to create the the predictions of the model externally and then just just put them in then that then you wouldn't need this type type of API but essentially it uses I think fit predict or predict probability for um for for this part

essentially and the the the clone part so on on the um whether you want to clone them or checking

Daniel Jacob: Um can I maybe ask um so I understand these two different classifiers as you already mentioned what we actually want to achieve is just really have to get a good um prediction um of the models but do we require any um consistency on the regressors so for example these are they're completely independent they can have different regressors I believe um and different regularization so even if the feature set might be the same they end being completely two different functions in the end. Is there anything that you guys observe like when you plug in the the the same set of features um that this somehow helps or because if I'm thinking back from the linear setting it would make a mapping um on on kind of always the same set of features on on X on Y and on D but here they are completely independent or does it doesn't it matter

01:51:48

Sven Klaassen: Oh, essentially here in what how we implemented they're not usually completely independent. So, so essentially because of the the the the the um if we go back for for for the the causal problem um in a very stylized version right of of this graph, we want to consider all the confounding factors X or common causes in this the adjustment to to close all back door paths but essenti There are some variables you could potentially just put into the outcome regression or in the treatment regression, but that usually doesn't make then a difference if you just in increase the set X um by one or two variables additionally. So so in our case if we go to the the method we just set one feature vector X which is used for both right. So, so it's used for treatment and for outcome regression usually and our idea is that these are the features which are um or mainly the features which are relevant to keep things comparable in the notebooks. You will see what happens maybe if you add one feature which only affects then the treatment and only the outcome or things like this but but the main idea is that that they will be used for both

01:53:12

Daniel Jacob: But takes.

Sven Klaassen: both examples and the go the goal is your correct iss essentially just get sort of a good predictive power for for this this pro these problems. So you we will

see that if we use random forest boosting in this example won't change the result that much. If we have this smart um

Martin Bierey: Um just a related question maybe to this. Um so I understand in the so in the model where we basically predict the propensity to be treated that makes a lot of sense for me that I basically go for the model that's explaining most part of the variance. Um but in the second model where we predict the outcome, this is not so clear for me there because we don't have the treatment variable in there, right? And we would still expect that the treatment explains a large Yeah.

Sven Klaassen: It it depends a bit on the problem and some some you you have the treatment in there usually for example in the interactive regression problem like this was just a general description it's an outcome regression what actually is in the outcome regression slightly differs in the partially linear you don't need it because of this additive structure again but in the the interactive one essentially there are two models we we are using a t-learner in the background to to fit one model for the untreated

01:54:31

Sven Klaassen: and one model for the treated. So it's it's in principle that the the the treatment is in the outcome regression also included in these examples. So so you're correct in most cases in the outcome regression you should include the treatment for for these examples. That's that's correct.

Martin Bierey: Okay. And then in both models you would basically aim for high accuracy.

Sven Klaassen: Yes. Yes. Um high accuracy and Yeah. Uh one part we will see in the tutorials what what might happen if you have something which only helps you to predict your treatment then you might not want to include it right because it could increase variance of of your estimation it's quite simple but the standard structure so we have defined our model what are our treatment variables what are the variables we want to include in the regression or classification problems That is all included essentially in the data, right? So what is X, what is Y, what is D? And then we said what what regressions do we want to use?

01:55:37

Sven Klaassen: For example, here the there G is the outcome regression. M is the

treatment regression or classification. In this example, we just plug in the the the corresponding machine learning algorithms either for XG boost or random forest for both of these. um then we can add some hyperparameter tuning. I will say something about how we do this afterwards a bit in a bit more detail how to do it. But the basic idea is that we have a tune models method where you have to set up some hyperparameter space and then it just tunes your hyperparameters. that that is sort of so the idea to just try and the the underlying concept it's what you exactly said is we want to just improve predictions or get sort of the best predictive performance of course cross- fitted performances right um predictive performance for the the prediction of the treatment and the the outcome or in our regression and classification problems for both of these and um this updates I said the the hyperparameters here here we just did 10 10 runs to to keep it fast for for the slides in this example to to to render the slides quite fast.

01:56:56

Sven Klaassen: And then the the final sort of part was essentially the the or to to run it. We we wanted to be somewhat close to the skarn style API to have essentially to have a fit is you you set up sort your learner your problem saying what is my treatment what is my outcome and then with fit all these models are actually fitted right these these machine learning models before it's just hyperparameter set it set set the hyperparameters increase them and with fit you you run the the the model and this gives you uh internally does the crossfitting to relate repeat it again to the these part the the previous examples. So the tuning and the learners help you to get somewhat good predictive performance on these regression and classification problems and in fit internally it does some some cross fitting right just runs the models like on different folds get the predictions collects them builds a large vector of your predictions large score vector and then solves these types of um score problems essentially to give give a final um point estimate and in this case for what we can see for example if we run a problem with a random forest so this was a model which we defined with random forest we get a point estimate of eight as an effect of 8,135 right but of course the standard errors in these problems are quite large because there's generally a lot of noise in

01:58:27

Sven Klaassen: these measurements right but what we have in these these financial estimates and the same thing if you run it for for boosting In this example, if you use the boosting part, both of them are seen like very simply hyperparameter tuned, you get quite similar estimates, right? In a very close range of these examples. So, so the the idea is what what actually should m should only matter in the background is sort of the predictive performance to estimate these these conditional expectations or probabilities in the end which are used to solve these problems. And then it depends a bit on your the model you're plugging in, right? But essentially if your model is performing quite well the these estimates because then your predictions should be quite close right to to each other then the predictions should be also very the the final point estimate should also be very close and um overall this the object has a bit more information right. So, so in a summary you or with if you print it you can see a whole summary what is essentially how many observations were used or what what the data looked like and then which which type of machine learning algorithm was was used to fit the corresponding regression or classifications and what the corresponding out of sample performance was.

01:59:55

Sven Klaassen: And in this model you can see that these are two models for the outcome regression and this is the t-anner structure we are using right. So so this is the the out of sample performance for the untreated or controlled units and this is the out of sample performance for the treated units in these in this part and yeah just just as a summary for both of these. Yeah, here it's a bit more more detailed on which which parameters we used. Okay. And the final part just and of course you you you you can yeah like I said there are also some other options to evaluate the the the the performance of the learners but but yeah you you we I don't want to go go into detail there now just the final point is that of course in the summary you get your typical types of summary for standard errors t values p values and confidence intervals but you of course you you can also um create confidence intervals at different types of levels, right? If if you want to and these are in these cases are done then with these by this framework at the end via this normal um approximation essentially.

02:01:10

Sven Klaassen: so valid approximation but this allows you to also include jointly valid confidence intervals by doing some sort of bootstrapping procedure for for for if you have multiple in the different diff literature if you have multiple um treatments at this effect estimates at the same time. So any questions on the the general idea of of of the workflow how the the typical steps are. So we have to define the model the learners you have to fit or tune hyperameters and and fit the models detail. If not then I would change or go to the the essentially the version on the hyperparameter tuning. So um this is a quite recent addition to the package. So, so we try to integrate the Optuner tuning framework in in the package because it's quite flexible and offers a lot lot of customization options but I think it's also quite straightforward to use. So the the idea is of course that the the only part which is actually up to the user or to some degree is to get this good predictive performance. Right?

02:02:29

Sven Klaassen: So the scores orthogonality part that's that's based that's based on the model. The cross fitting is also implemented. So the predictive performance in these examples is the thing you what what you can actually um what what matters and what you can actually um change and improve. And the the the um part is that this affects directly the the bias and specifically also standard errors sometimes to a large degree. One can see of course some default values some for some learners. I don't know if you know for some default values for example from sklearn if you run a random forest in skarn and things like this with leaf sizes they are sometimes very small. So the question where how well your model generalizes if it's overfitting sometimes really a lot. So so and then all of these hyperparameter tuning parts become relevant for for for for these these models and um so the goal is to find these hyperparameter configurations that minimize out of predictions. So the nuisance errors so that they generalize well and we are doing this via optuna and the the basic idea is that you set up if you run certain learners in this example we will use light GBM as a boosting algorithm for this and in principle if you just in this problem um if we were using a very basic problem where with a generated data set so so if you want to look at the det details.

02:04:09

Sven Klaassen: So it's here for for the data, but it's just generated 500 observations. It's internally um a quite simple model. It's it's linear and a logistic assignment of the treatment. And if you run in this example, if you run a light GBM regressor and classifier without any any hyperparameters, you get an estimate of 06, which you will see is is uh quite quite bad. and a a a somewhat somewhat large standard error which we can compare. So this will be our untuned model and um instead what you can do for for for the hyperparameter tuning you you have to define with optuna you have to define trial objects to say okay in my hyperparameter space what are the hyperparameter ranges I want to to to test and these this this is so you have to define a function which returns a dictionary and the dictionary has to match the hyper parameters you want to choose and you're you are running certain tri these trial objects with Optuna which which you have suggest either a floating value of this in some space between 0.01 for for for the learning rate or 0.1 and it's distributed not uniformly where it's sampled but with a logarithm the the logarithm is distributed uniformly.

02:05:42

Sven Klaassen: So, so you sample over the larger just essentially the range larger if you have a large range to see in which range your your learning rate should be. So there are different types of suggestions and these these are the hyperparameter spaces you you have to define for your learners and it's specific to what what machine learning algorithm you're choosing right so so if you're choosing random forest might look different right you don't have the the lambda values but essentially you have something like the minimal samples per leaf you have something like the number of trees and things things like this so so all of just the the standard plug-in parts how to tune your machine learning algorithm of these parts and you you can define different hyperparameter spaces for your um treatment and outcome regression. In this case, we just reusing the same one, right? But the here they are identical but you you have to define the different uh you have to define hyperparameter spaces what you want to ch where you want to look at these data and typical or the the search spaces are quite easy to define.

02:06:48

Sven Klaassen: So there are you can suggest either floating values uh integer values or categorical. So so if you have at least some ordering so to optimize these things or categorical values. So so you have different for for example here with a support vector machine for this one if you want to use different kernels for these examples. And um yeah there are a lot of options you can choose. So you you can have instead of usually uniformly sample from these hyperparameter combinations, you can also have of course some sort of step size. And how is this actually used now in double ML? Um yeah, you set up a hyperparameter space and you have to define for your learners for each learner what is the corresponding hyperparameter space I want to I want to um test want to optimize over. Right? So, so for my outcome regression, I define my hyperparameter space and for my treatment regression. And afterwards, you have to set up some general settings. Think okay, how many trials do I want to run my examples?

02:07:59

Sven Klaassen: How many different trials do I want to give my algorithm to to to optimize these hyperparameters? And yeah, whether I want to see progress bar, things like this. But essentials just for this example and of course there are some some logging things going on and that's it because the next part would be so so now we define the same object again and the next part is to run the tune models method with the and defining the the hyperparameter space and the using the same settings. And what happens now in the background is essentially that different model these different models are fitted and also not on the same set but on diff different different version of cross validated sets to to avoid overfitting. All these type of models are fitted and optimized. And the nice thing about Optuna is it's not random grid search, right? It's it's um it's of course it's also not Beijian hyperparameter optimization, but it's it's approximating Beijian hyperparameter optimization as a go with a goian. So it's not a goian process, but a three parts structure.

02:09:08

Sven Klaassen: So essentially it's trying to sample from the hyperparameter configurations to find some balance between exploration. So sample on the the the

hyperparameter space and if it finds good approximation a good combinations to sample in the neighborhood around these good combinations a bit more than around other neighborhoods. So it's already like I said it's not grid search and it's so so it's bit optimized on how to actually find the good hyperparameter combinations but that's part of this optuna package and and the idea is that we just sort of um um sort of pass through the values in a good way to to have these models specifically or separately trained because for example you you see in the the interactive model we had this t-learner structure we have one model for the treated and one for the the untreated. So we are also internally just just uh finding the right observations try on which one we actually have the the outcomes on which one we can actually tune and all of these type of things. So, so, so to to just enables an unified interface for this and um then after the tuning you can just call fit again and um because the tuning will usual you you can will set the hyperparameters then to to the the the best chosen values.

02:10:36

Sven Klaassen: And in this example, you can see the decim changed a lot from roughly zero or close to being zero to 0.5. And if you compare the result um we will compare the results at the end to see how how this actually affected things and you can see that the standard error was is is much smaller. This really depends a bit on the problem how many how much noise there is and how much you can improve on the fit by by tuning these parameters whether the improvements are large and just just to give some idea what what is happening in these these examples um I said the this interactive regression model has this t-learner structure right it's internally creating two outcome regressions one for the treatment group and one for the control group. So we have the the the main argument for this is um for the the um for everyone who doesn't know what the difference from this T and S learner is came up quite a lot. So if you have a binary problem and you run a regression and um just a regression and include your binary variable and X in this regression that's called an S learner because it's a single regression.

02:11:55

Sven Klaassen: you just run one regression function, but you you include the variable the the binary treatment as a regressor. And a t-learner is doing just two regressions, one for the treatment group and one for the control group. And this would be the same as

running a single regression but interacting each feature with the treatment. Right? that in a linear problem you could just create all the interactions between treatment and features like $D * X$ right so so you have two separate models one linear model for the treated group and one linear model for the control group so the T-learner is a bit more flexible that is the idea so you have sort of separate learners as you you don't it's a bit more flexible and saying what is the how the treatment group behaves and how the control group behaves and in this case what we All we're doing this quite often. So we're running quite often internally a regression only on the control group and only on the treated group. And these are implicitly created just via this input argument.

02:13:03

Sven Klaassen: So they they are just a cloned version but they're just fitted separately. It's a cloned version of of the outcome regression but fitted separately. So so it's one outcome model for the control group, right? And one outcome model for the treatment group. It's just use using the same input parameters for the the the learner, right? Both using, for example, boosting, but it's just two different models, but each one is tuned separately because sometimes you have unbalanced treatment assignment, right? So, so you might have quite different hyperparameters for each of these models. And um in the current version what we've seen they're using the same hyperparameter space but get different combinations but you can in principle you can give each of them their own hyperparameter configuration and yeah the these hyperparameters are set then at each crossfitting fold because in principle you could also set different hyperparameters at each fold but I think that's really not a good idea but it's just so for example for this treatment um The control group for the regression or treatment is if there are multiple treatments that might also have multiple treatment version.

02:14:17

Sven Klaassen: You can see the learning rate is set if um the learning rate is set to this value 0.01 right minimum child samples lambda and then afterwards we can also see that this is quite different for for the outcome regression for the control group which has a much larger learning rate in this example. Um, yes.

Ani Pole: Sorry, I I have a question about the t-learner. So, basically, we're going to have like three models in this case, right? One for the treated, one for the untreated, and one

for the treatment effect. Uh, and then how do this all come together first?

Sven Klaassen: Okay.

Ani Pole: And the second one is like why do we need to um fit those

Sven Klaassen: Um

Ani Pole: um two first two models separately and not just one.

Sven Klaassen: so so essentially we don't need to the first two models we don't need to fit them separately. That's just how we implemented in the package to make it a bit more flexible. So, so you could run one model which uses its features the treatment and the features X, right?

02:15:26

Sven Klaassen: So, so all D and X and then um create the predictions for for the control and the treatment group. But this often I think for the this is called then the this slearner type. So for single regression, but essentially this is um I think often not as flexible and we've seen sometimes that that this the the the effects or the the predictions if you don't if you have treatment effects it's sometimes hard to capture some heterogeneity in the these treatment effects because the other features X might be more relevant not for the treatment effect but for the outcome. So it starts to split or starts to to capture the effect quite um at only at quite small levels for the treatment. So what we are doing we are fitting two outcome regressions to make models more flexible. The the idea would be if in a linear regression you you can have for two different options. You just run a linear regression where you just use the treatment as one regressor. So, so one additive effect or you just run a treatment uh run a regression where you include all interactions between treatment and other controls.

02:16:41

Sven Klaassen: So, you have like all these interaction variables. So, in principle you have like two separate linear models because you're switching just one each control on or off, right? The treatment times the control is $D * X$ is either zero or X depending whether D is one or not, right? So, so and if you're including all of these that you you and this is sort of the t-anner structure you have all these interactions available so it's more flexible and this this is why we are choosing this type of structure so it works not only with the linear model but with something like random forest boosting things like this and um so so and

how these come together um we let me just share the documentation. We abstract it away a bit away from this because I don't know. Can you see? No. No, it's not.

Jan Teichert-Kluge: Almost done the

Sven Klaassen: Yeah.

Jan Teichert-Kluge: slides.

Sven Klaassen: Uh, sorry. Um, or let me just go to the documentation because that gets quite complicated. Essentially, how they affect these things.

02:17:59

Sven Klaassen: It this is specifically in the scores. So, so we have for each of the models there's a specific underlying score which is solved and this score has this type of property and if you look at the interactive regression model so so the the one which we have right here let me increase a bit so here's also weighted version of this is that this is plugged into these type of score elements so it's we're using the outcome prediction so this is for the treated group minus the control group the outcome prediction and then you have an correction term which also depends then on the propensity score. So, so we are computing this type of vector and this is completely different for each model right for each model it looks or it's quite similar but but different the these are plugged into these type of scores and these scores have this orthogonality property essent and this with this property you um and and then this is solved for let me zero right so this is linear so so so it's it's solved so it's a moment condition So we're solving a moment condition for this example and this just depends on these plugged in machine learning parts.

02:19:16

Sven Klaassen: So so for but for each of these models it's this is slightly different and for for the partially linear just to refer to the previous point this depends then on residuals this moment condition there you just get residuals on residuals regression which you can express as a moment condition which is solved right and and and then then you get get the same same argument and this therefore we we need we don't or we have these three types of plug-in parts which which how they how are then they are then used to to finalize our estimator in these examples. Okay, great. But the point is still valid in principle. You could run just a single regression but using this as a feature the

the treatment and then then you you could also plug these all these estimates in. But this is sort of is a design choice in the package to have like this more flexible version available. And in this example if we compare the tuned versus the untuned example you can see that the effect changed a lot right from 0.06 06 to 0.46 4 six and because it's a data generated process right where the of course because of this linear form the default values didn't work as well.

02:20:41

Sven Klaassen: So there essentially you can see the untuned version in this part has a really huge um uncertainty assigned to it because we're using the standard light GBM algorithm to to solve this linear problem right and if we tune it a bit in this example this in this this example it already makes a huge huge difference in in the the the how the confidence interval and how we estimate it right the the uh the the point estimate for this example Um, yes, Martin.

Martin Bierey: Um could you give some intuition on how you derive the standard errors here? Does it more come from the different folds on which you are doing it or does it come from how good your score function works in of organizing or or where does it come from?

Sven Klaassen: um they they the standard errors they they they come from the the um it's a walt type version. So essentially they they come from the score function right. So so this we are solving if we're going back to the the the the documentation we we're solving. So this thing we I I think it's a good good to discuss this in detail.

02:21:51

Sven Klaassen: We we try to abstract a bit because these formulas get quite complicated quite fast. So so that's the main reason why we tried not to include as much. Essentially you have this type of score function you are solving and this will give you we go in the variance estimation part you you you get a formula for specifically on the variances how they depend on the score because this is you have an asymptotic um so you're line you're linearizing your estimator and you you get a linear representation of this and if you have this linear form which converges to your asymptotic normal distribution. You can also compute the um in this linear form specifically the the standard errors in the same fashion. So what we are doing is we we we're computing

the the second moment of the scores empirically and we are needing need also this type of scaling factor which is depends also a bit on on the problem and the score but really it it really is sec in principle it's a second moment of the score which we are computing for for for for this time okay I hope this this one more here to see how how we get this this type of standard errors.

02:23:12

Sven Klaassen: And we we've we test coverage for these examples with with data generating processes is to to see whether these actually covered these effects with close to 90 95 depending on what level you choose. So that the and this usually performs quite well. And you can see in this example what happened is a lot if you didn't tune it your confidence interval got really large because you had large bias that that can sometimes happen on whether you have a lot of noise in your problem there's a lot of uncertainty sampling uncertainty or like that could also be done via your your learners essentially if they they are not if they have huge type of variation there. Um, so if you want to take a look at how the learners performed on on the the these samples, you you can use the the evaluate learners method which calculates you can say the metric you want to use, but this per default uses root mean square error for continuous um variables and log loss for binary ones as as a default option. And this just computes the cross fitted performance of your learner.

02:24:27

Sven Klaassen: Right? So, so also out of sample predictions on the cross fitted versions and you you can see in these examples that the the these out of sample performances improved a lot for these examples. So this is the reason why the estimation was much better in theam what we've seen of of the the causal effect in this example. Right? So coming back the main point is we need good predictive performance or it's always better to have the the good predictive performance for these ex and so so for the the last part I don't want to go into detail in all of these slides but I think it's really helpful to to see something which is a bit more can be I think applied a bit more generally is you can run all of these parts essentially with with pipelines in scikitlearn or things like this because in scikitlearn it's a nice thing right pipelines are also estimators if you have this type of pipeline structure so what you can do is for example in these pipelines you you

can run you can combine two base regressors for example a linear regression and some boosting algorithm random forest of course all of this the learning part is the long computationally intensive part right the machine learning model.

02:25:52

Sven Klaassen: That's the the main bottleneck for computational time. And then you you can it's a you can just stack them with this scikitlearn element, the stacking regressor, right? You you can just combine both of the learners. Um fit the model separately and find a combination of these models via final rich estimator, right? And then you build a pipeline where you for example also before just add a robust scalar before fitting your model right in these examples. So you have you rescaling your model or you can use a standard scaler whatever you want then fitting the stacked regression problem. You can run the same thing with the classification part, but then you just have to use for notation. Um the the um oh I think there's something oh no it's correct. I I just saw I didn't see that the the bracket started here. Um so so essentially you just have you use the notation for the scikitlearn and how the scikitle learn part in adjust addresses all the elements of the stacking regressor and things like this.

02:27:07

Sven Klaassen: So just with the underscore notation and things like this but you can tune then separate components. So for example this one tunes your your boosting part and this the other one tunes your rich regression at the end and things like this. all these combinations and um just specifically you can also set some learners to a smaller number of trials if you want to use if you say for my outcome regression I want to tune as much I want to tune more the other regression you you can combine combine these approaches um yeah like I said I don't want to go into that much detail in these examples but this just the idea you you can use all of this pipeline structure in the same tuning form everything works in the same fashion because I think for for for very robust estimation it's often very helpful to include for example a linear model some nonlinear model things like this and then to see which one actually gives best predictive performance right these examples and um if tuning is done you you can also take a look in at the detailed tuning results which might be sometimes helpful if you if you if you want to find out why something is looking not as good, why why some hyperparameters

are chosen.

02:28:30

Sven Klaassen: So So um if you want to take a look at tuning results, you have to set the flag for returning the tuning results specifically. So it's not only modifying your object by setting the hyperparameters, but it's returning then an object which includes all of the the overall tuning results that that are included just by via this flag. And in this example, you see we we've run 50 trials on on the the outcome regression for for the control group, 50 trials for the treatment group, and 100 trials in this example just just for the propensity score. Again just as a basic example and and you we we get this this type of estimator. This this was with the pipeline right running both for both the certain pipeline examples and yeah it becomes a bit more in detail but essentially you can you can access all of the results separately like in the results you have one specific tuning result for your outcome regression for the control group one for the treated group right and one for the propensity score in this example and if you want to look for example at the propensity score you see okay what what is the best score that was achieved the corresponding because sklearn has scores are always um scores are always optimized so it's a negative loss right because you want to optimize it and you can see all the parameters that that were chosen for these examples and you can even take a look at

02:30:06

Sven Klaassen: all the specific trial data right like um how long long the trial took what the actual best loss or score value was you've seen for all of these examples and and you can visualize all of these settings also and here you can see which is I think quite good that it really tries to to optimize step by step of these comparisons so any questions regarding this I don't want to go through all these combinations you can have the standard plots on different hyperparameter combinations things like this right what what hyperparameter combinations were the the ones that were tried and chosen and which got uh which which um left a certain certain objective value. So, so overall essentially the the the tuning ML models I think is just an hopefully good way to use um very or or very flex is still in some settings simple but flexible way to tune all these specific models in these hyperparameter combinations to to get actually sometimes much better results on on and much better um or smaller standard errors on on the

predict on the effect estimation and the main part is that what as a as a user one has to say okay what are is the hyperparameter space I want to check I want to check out where where I want to optimize on how many trials do I want to choose and then this tries to

02:31:35

Sven Klaassen: to um which parameters are um optimized separately right and so so just for for more details uh we have a detailed part also in the documentation if you detail example but we will see one example how to use it also in the notebook session. So any questions on this or general to the general idea on Yeah.

Chieri Naito: Yes. Uh thanks. So I think Tian already mentioned but um yes. So yeah I I understand that the hyper parameter tuning is very important but at the same time you know um this encoding part is also very uh important for the model performance. And uh for example you know um cat boosting uses cut uh um target encoding a specific target encoding that you know gives uh good performance but uh I think we we were using um uh the different package and the difficulty was that they don't because it's not a scalar model uh you know we couldn't just uh use you know as is uh but uh this package had uh like basically you know u I'm I'm like talking about specific case but we were using the coal forest and then basically you know if we used generalized uh random forest in the second stage that's okay so what we did is um you know first stage we just do our you know coding you know cat boost and then uh we can also So add hyperparameter tuning you know um using optuna and then you know pass the um uh you know results to the generalized

02:33:30

Chieri Naito: random forest but uh yeah I I guess um in this package uh it's not possible right like for example cat boost is it possible to uh do similar hyper tuning as is or if not um maybe We would like to know how we can integrate it uh in a detail.

Sven Klaassen: Um I I'm not quite sure the maybe I'm not that familiar with the the part on um K cat boost what what how the the hyperparameter or or what what parameter for feature transformations actually were used right because of I think you you you can use cat boost in in in the ML package al sure you should also be able to use cat boost but of course like if you want to have very specific hyperparameter transformation that

is not cannot be straightforward applied except if it fits in this type of pipeline structure, right? Because what is we're doing at the end is we're fitting an estimator and the pipeline structure just ensures that that we have all these estimator properties available, right? The the fit sort of a predict method which runs the whole pipeline just once um from from the original data.

02:34:47

Sven Klaassen: Um what would be possible is to to generate these predictions externally sort of separately and then supply them to the model saying like if you have a very specific model where you want to have a quite different predictive um pipeline for your propensity score. You can just predict your propensity score externally and just put in a vector in your fit part to say okay this is my vector of estimated propensity scores. treat them essentially as we you would have estimated it with some other other models. So so in the same same form that would be one option. Um so regarding the part on on causal forest so causal forest or as a comparison to the package causal forest is um quite similar for to the interactive regression model. So the internal part on what is estimated on the score for causal forest is very close or very similar to what is estimated in this interactive regression model. Um yeah it's not here like but in if I also go into the the this uh what I said in the score function in the interactive regression model so for co forest is is very closely estimating thing like this which is um just with certain forests and then the splitting is a bit bit different right to to to find some heterogeneous features right so so for for this interactive model is very close to what coer forest does but cos forest on The predictions then gives point predictions for certain values of X combinations,

02:36:30

Sven Klaassen: right? On on what the effect looks at a very specific combination of all these X values and I think what one here or what one then should do. So in these cases I think both of them should be very similar if you put you use the same predictive model which gives the same predictive performance. Um but if we come to the group or conditional average treatment effect estimation then I think sometimes it's not very useful to have the exact point at all the features X but maybe you only want to have marginal conditional on age on one or two features right not not all the whole whole

whole um vector of all the features you have. I think renov has some options for this but you would have to to add it separately. um or in in some one or two steps but but I think this that is in this case the more more uh or other difference and for all the other models that might be sometimes then a bit different to to the cause of forest approach. So what I I mean is in one model there's a lot of overlap but but causing forest is for example not straightforward applicable to diff estimation right and and things similar things okay and yeah so this was already a bit more than than the other part but regarding on on the feature part I I don't know it really depends a bit on the features how how you construct them if if you have more detail or you can

02:38:05

Sven Klaassen: still um or we we discuss it separately. So so I think so it's now the good point for to to go into the the the lunch break right on on the if we take a look at the schedule um like I said if you open the the note not notebooks you you can just save them or say save link as and then then you should be able to download it. MS the the the first one not the different diff will be

Tsz Yan Lam: Are we going to use the first one or the second one today?

Sven Klaassen: in the next session because we will then make the definitive diff explanation and assumptions before and then go through the different and today will be return on advertisement spent um in in this this example.

Tsz Yan Lam: So just be clear,

Sven Klaassen: Okay.

Tsz Yan Lam: we we just need to download the notebook. We don't need to do anything else. Is it correct?

Sven Klaassen: Yeah. So, so the idea is um so the notebook will look like this.

02:39:04

Sven Klaassen: So you can so the idea is for for you also in the break is maybe to download the notebook and try to maybe already go through some parts and what what you understand what you don't understand because the how we hope the sessions will go. So, so Yan and Yan will do the sessions on the notebooks and uh in in like two groups and the the how we hope this goes is that you go step by step through the case and discuss what you think of the example, what what you think uh or if everything sort

of is clear and they will explain sort of step by step what they are doing and what what the notebook is doing. And of course if you have some if you run the notebook yourself and you see already what what you did understand what you can change how how things are affected I think it makes more sense then just if if you just um follow them on the screen specifically if it's code right so so so um then then just like take 10 20 minutes to at first maybe prepare or just on on the notebook and these example

Tsz Yan Lam: Got it. So we will be coming back at 1:30 on this call and then we will create big our rooms to assign people relatively randomly by me uh into the the different different groups. Is it okay?

Sven Klaassen: Yes. Yes. Great.

Tsz Yan Lam: All right. Thank you very much. Anyone have any question before we start our lunch break and you should look at the notebook. All right then. See you in an hour or so.

Transcription ended after 02:59:16

This editable transcript was computer generated and might contain errors. People can also change the text after it was created.